People's Democratic Republic of Algeria
Ministry of Higher Education and Scientific Research
National Polytechnic School of Oran (ENPO Maurice Audin)



Master's Thesis

Specialty: Information System

Option: Engineering and Management of Information Systems (IMSI)

Communication Optimization for a Decentralized Federated Learning System

---

Presented by:

Mohammed Salaheddine Belarbi

Supervised by: Dr Nawel Bendimerad
President: Dr Asmaa Insaf Djebbar
Examiner: Dr Samah Bouamama

Academic Year: 2025–2026

Introduction

Federated Learning (FL) has emerged as a promising paradigm for training machine learning models across distributed devices while keeping raw data local. By exchanging only model updates instead of private data, FL addresses critical privacy and regulatory concerns that arise in centralized learning. However, classical FL relies on a central server that coordinates clients, aggregates updates, and redistributes the global model. This centralization introduces a single point of failure, limits scalability, and creates a communication bottleneck, especially when the number of participants grows or when devices operate under constrained network conditions.

Decentralized Federated Learning (DFL) removes the central server and replaces it with peer-to-peer or hierarchical coordination mechanisms. In such systems, participants exchange model updates directly or through elected intermediaries, and global consensus is achieved through iterative local aggregation. While DFL improves resilience and scalability, it exacerbates the communication problem: each participant may exchange large model updates with multiple neighbors, and the absence of a central coordinator makes efficient scheduling and redundancy control more difficult. Consequently, communication volume, latency, and bandwidth consumption become first-order design constraints.

This thesis addresses the communication bottleneck in a specific class of DFL systems: a hierarchical, sharded architecture coordinated by a Directed Acyclic Graph (DAG) and protected by zero-knowledge proofs (ZKPs) and cryptographic commitments. The system under consideration was originally developed in the author's Graduation Project (PFE). It organizes participants into a Client–Shard–Global topology, records verified updates in a per-shard DAG, and enforces integrity and Byzantine resilience through polynomial proxies, commitments, and multi-stage robust aggregation. Although the system provides strong correctness and security guarantees, its communication cost remains high because every client transmits a full-dimensional, full-precision model update to its shard chief in every round.

The central research question of this work is therefore the following:

> *How can the communication volume of a verifiable, sharded, DAG-coordinated decentralized federated learning system be substantially reduced without weakening its existing integrity, verification, and robust aggregation guarantees?*

To answer this question, the thesis proposes a communication-enhanced architecture that inserts a carefully designed gradient compression layer at the client-to-shard-chief boundary. The compression mechanism combines top-magnitude sparsification with optional fixed-point quantization and client-side residual/error-feedback accumulation. Critically, the design preserves the existing zero-knowledge verification pipeline (proofs are generated on the original dense update), redefines commitments to bind the compressed payload, and reconstructs a dense vector before robust aggregation. In addition, a Com-

munication Metrics Logger is introduced to measure real serialized byte counts rather than relying on architectural estimates.

The main contributions of this thesis can be summarized as follows:

1. A compatibility-preserving compression insertion pattern that reduces the dominant communication term (the model update) while leaving the zero-knowledge proof, polynomial proxy, robust aggregation, and DAG recording layers logically unchanged.

2. An analytical communication model and a set of formal requirements (R1–R6) that any optimization of the base system must satisfy.

3. A measurement strategy based on real serialization at the network boundary, enabling reproducible quantification of compression ratios and absolute byte savings.

4. An experimental evaluation plan that quantifies the resulting accuracy–communication trade-off under non-IID data distributions and asynchronous operation.

The remainder of the thesis is organized as follows. Chapter 1 presents the theoretical background necessary to understand distributed machine learning, federated and decentralized federated learning, peer-to-peer communication, asynchronous operation, DAG coordination, and the principal techniques for reducing communication cost. Chapter 2 reviews the state of the art, critically synthesizes existing approaches, identifies the research gap, and positions the present work. Chapter 3 formalizes the communication behavior of the existing system, states the requirements that any optimization must satisfy, and describes the proposed communication-enhanced architecture in detail. Chapter **??** presents the concrete implementation of the compression layer and the metrics logger. Chapter **??** reports the experimental evaluation. Finally, Chapter **??** concludes the thesis and outlines directions for future work.

Contents

1 Theoretical Background **9**

1 Introduction . . . 9
2 Distributed Machine Learning . . . 9
2.1 Definition . . . 9
2.2 Centralized versus Distributed Learning . . . 9
2.3 Distributed Training Process . . . 10
2.4 Advantages and Challenges . . . 10
3 Federated Learning . . . 11
3.1 Definition and Principles . . . 11
3.2 Federated Learning Workflow . . . 11
3.3 Federated Averaging . . . 12
3.4 Challenges of Federated Learning . . . 12
4 Communication in Federated Learning . . . 12
4.1 Communication Model . . . 12
4.2 Rounds and Communication Frequency . . . 12
4.3 Volume, Message Size, and Message Count . . . 14
4.4 Latency and Bandwidth . . . 14
4.5 Communication–Computation Trade-off . . . 14
5 Decentralized Federated Learning . . . 14
5.1 Motivation and Definition . . . 14
5.2 Centralized versus Decentralized Coordination . . . 14
5.3 Decentralized Aggregation . . . 14
5.4 Communication Challenges in DFL . . . 15
6 Peer-to-Peer Communication . . . 15
6.1 P2P Definition and Characteristics . . . 15
6.2 P2P Topologies . . . 15
6.3 Neighbor-Based Communication . . . 15
6.4 Gossip Communication . . . 15
7 Asynchronous Federated Learning . . . 16
7.1 Synchronous Federated Learning . . . 16
7.2 Asynchronous Federated Learning . . . 16
7.3 Synchronous versus Asynchronous Operation . . . 16
7.4 Stale Updates and Asynchronous Decentralization . . . 17
8 Graph Theory and DAG-Based Systems . . . 17
8.1 Graph Fundamentals . . . 17
8.2 Directed Graphs . . . 18
8.3 Directed Acyclic Graphs . . . 18
8.4 DAG Coordination . . . 18

9 Communication Optimization Fundamentals 18
9.1 Communication Efficiency 18
9.2 Fundamental Metrics 18
9.3 Metric Relationships 19
10 Model and Update Compression 19
10.1 Quantization 20
10.2 Sparsification 20
10.3 Pruning and Low-Rank Representation 20
10.4 Selective Transmission 20
10.5 Compression–Accuracy Trade-off 20
11 Convergence and Communication–Performance Trade-offs 20
11.1 Consensus and Optimization 20
11.2 Mixing and Convergence 21
11.3 Communication and Convergence Trade-offs 21
12 Conclusion 21

2 State of the Art **22**
1 Introduction 22
2 Communication Challenges in Federated Learning 22
3 Communication-Efficient Centralized Federated Learning 23
3.1 Reducing Rounds and Selecting Clients 23
3.2 Gradient and Model Compression 23
4 Transition to Decentralized Federated Learning 24
5 Communication in Decentralized Federated Learning 25
5.1 Gossip and Decentralized Optimization 25
5.2 Asynchrony, Peer Selection, and Topology 25
6 Communication Optimization Techniques for Decentralized FL 25
7 DAG-Based Federated Learning and Communication 26
8 Detailed Literature Review of Representative Papers 26
8.1 ProFe 26
8.2 GossipFL 27
8.3 MIMO Over-the-Air Computation 27
8.4 MAR-FL 27
8.5 BCFL 27
8.6 Asynchronous Approaches: DRACO and FedDual 28
8.7 Reception- and Age-Aware Aggregation 28
8.8 PISCO 28
8.9 AIRPLAN 28
8.10 Quantized Rank Reduction and OCD-FL 29
8.11 Graph-Based Gossiping and DAG-AFL 29
9 Cross-Paper Critical Synthesis 31
10 Comparative Analysis of Existing Approaches 31
11 Critical Analysis and Research Limitations 32
12 Research Gap 32
13 Positioning of the Master's Thesis 32
14 Conclusion 33

3 Proposed Communication-Enhanced Architecture 34
1 Introduction . . . 34
2 Summary of the Existing Decentralized Federated Learning System . . . 34
2.1 System Topology . . . 34
2.2 Per-Round Communication Flow . . . 35
2.3 Implementation-Level Observations . . . 37
3 Communication Model and Requirements . . . 37
3.1 Notation . . . 37
3.2 Message Composition in the Base System . . . 38
3.3 Total Communication Volume . . . 39
3.4 Message Count . . . 39
3.5 Requirements for the Proposed Architecture . . . 40
4 Proposed Communication-Enhanced Architecture . . . 40
4.1 Overview and Design Rationale . . . 40
4.2 Scope . . . 41
4.3 Architectural Components . . . 41
4.4 Compression Procedure . . . 41
4.5 Placement in the Client and Shard-Chief Pipelines . . . 43
4.6 ZKP–Commitment Soundness Argument . . . 44
4.7 Communication Complexity Analysis . . . 44
4.8 Communication Metrics Logger . . . 45
4.9 Compatibility with Robust Aggregation under Sparse Reconstructions 45
4.10 Trade-offs . . . 45
Conclusion . . . 46

4 Implementation — Communication Optimization 47
1 Overview and positioning . . . 47
2 Design goals and implementation constraints . . . 47
3 High-level implementation summary . . . 48
4 Detailed implementation . . . 48
4.1 gradient_compression.py — Module interface with residual support 48
4.2 Client-side changes (client.py) — residual state and compressed workflow . . . 49
4.3 Shard-side changes (shard_chief.py) . . . 50
4.4 System instrumentation (federated_learning_system.py) . . . 50
5 Mapping to Requirements R1–R6 . . . 50
6 ZKP-Commitment soundness and Invariant I . . . 51
7 Testing and correctness checks . . . 52
8 Experimental driver and reproducible workflow . . . 52
9 Security, auditability and ethical considerations . . . 52
10 Limitations and planned extensions . . . 53
11 Practical instructions for reproducing results . . . 53
12 Conclusion . . . 54

List of Figures

1.1 Overview of a typical distributed (federated) training round. Nodes compute local gradients on their private data (1), send them to the server for secure aggregation (2), receive the updated global model (3), and apply the new parameters locally (4). 10
1.2 Comparison of centralized learning and federated learning workflows. In centralized learning, raw data is sent to a central server, whereas in federated learning only model updates are exchanged while data remains local. . 11
1.3 Federated learning communication architecture. Left: clients with private local datasets communicate with a central server. Right: one communication round, including model broadcast, local gradient computation, optional compression, and aggregation. 13
1.4 Common peer-to-peer communication topologies used in decentralized federated learning: ring, full mesh, small-world, scale-free, directed acyclic graph (DAG), and star. 16
1.5 Comparison of synchronous and asynchronous federated learning. In the synchronous mode a global barrier waits for all selected participants, while in the asynchronous mode updates are incorporated as soon as they arrive. 17
1.6 Taxonomy of the main communication-efficient techniques in federated learning: quantization, sparsification, and hybrid compression methods. . . 19

2.1 FedSparQ's adaptive sparsification-quantization pipeline 24
2.2 Generic transaction/tip structure of a DAG-based FL system 26
2.3 ProFe's distillation and prototype-sharing architecture 27
2.4 PISCO's semi-decentralized communication protocol 28
2.5 Overall architecture of the AIRPLAN OTA-DFL system 29
2.6 Communication-cost vs. accuracy across topology families 30
2.7 Tip-selection and verification pipeline of DAG-AFL 30

3.1 Client-side pipeline showing the processing stages from local training to update transmission. Each client follows a strict sequence of operations: gradient computation, clipping to norm bounds, polynomial proxy generation, zero-knowledge proof computation, optional compression, and commitment generation to ensure integrity of the transmitted payload. 35
3.2 Per-round communication flow across the three-layer architecture. Updates originate at clients, pass through shard chiefs for verification and robust aggregation, and culminate at the global coordinator for model aggregation. This cyclic pattern defines the baseline communication profile; the proposed compression optimization targets the client-to-shard boundary where message count and volume are highest. 36

3.3 Module interaction diagram showing the information flow between client.py, gradient compression layers, shard_chief.py, and system-level orchestration. The compression module and decompression module sit at the client-to-shard boundary, enabling measurement of real communication statistics while leaving downstream verification and aggregation untouched. 38
3.4 Shard-chief side processing pipeline, showing how the shard chief receives updates, decompresses them if compression was applied, verifies proofs, and feeds the reconstructed dense arrays into the robust aggregation subsystem. Decompression is the first step, ensuring that verification and aggregation remain unmodified and continue to operate on full-dimensional arrays as designed. 39
3.5 Compression procedure showing the step-by-step transformation of a full-dimensional gradient into a compressed sparse representation. The procedure includes residual compensation (blending the current gradient with accumulated error from prior rounds), top-k magnitude retention using the formula $k = \lfloor \rho \cdot D \rfloor$, optional quantization to reduce per-value bit-width, and packing of indices, values, and metadata for transmission. The serialized size includes all components: retained indices, quantized values, and parameter-size information. 43

List of Tables

2.1 Comparison of communication-efficient decentralized FL approaches 31

3.1 Notation used in the communication model. 38
3.2 Components of the proposed architecture, existing versus new. 42
3.3 Trade-offs of the proposed compression layer. 46

4.1 Mapping of implementation choices to formal requirements R1–R6. 51

Chapter 1

Theoretical Background

1 Introduction

This chapter establishes the theoretical foundations necessary for the study of communication optimization in decentralized federated learning systems. It begins by situating the problem within the broader context of distributed machine learning and classical federated learning, before formalizing the communication model that underpins the remainder of the thesis. Subsequent sections examine decentralized coordination mechanisms, peer-to-peer and gossip-based communication protocols, asynchronous operation, and the role of graph-theoretic and Directed Acyclic Graph (DAG) representations in distributed training. The chapter concludes with a discussion of the principal techniques employed to reduce communication cost and of the fundamental trade-offs that arise between communication efficiency, convergence behaviour, and model performance.

2 Distributed Machine Learning

2.1 Definition

Distributed Machine Learning (DML) is the training of a machine-learning model by several computational nodes that jointly process data or optimization tasks. Each node owns a local dataset and contributes to a shared global objective. Distributed optimization seeks a parameter vector that minimizes this global objective while distributing computation and, in some settings, keeping data at its original location [1], [2].

2.2 Centralized versus Distributed Learning

In centralized learning, a coordinator or a single training service collects the data or receives all intermediate computations and updates one model. In distributed learning, computation is divided among nodes and intermediate states must be coordinated. The distinction is therefore not only architectural: distributed learning introduces synchronization, topology, communication, and failure considerations into the optimization problem [2], [3].

A centralized system can simplify coordination because one entity determines the current model. A distributed system can use aggregate computational and storage resources, but the nodes may have different processing speeds, network capacities, data distributions,

and availability. Consequently, the optimization objective must be considered together with the cost of exchanging parameters, gradients, or other update information.

2.3 Distributed Training Process

A generic distributed training iteration consists of four steps (illustrated in Figure 1.1).. First, a node receives or selects the current model state. Second, it evaluates a local objective or computes a gradient using a local data subset. Third, it transmits an update or model state to one or more other nodes. Fourth, the received information is aggregated or used to update the local state. If the nodes synchronize after every iteration, the process is synchronous; if nodes continue using locally available states, it is asynchronous. Each node performs a basic local stochastic-gradient step on its own data. Distributed

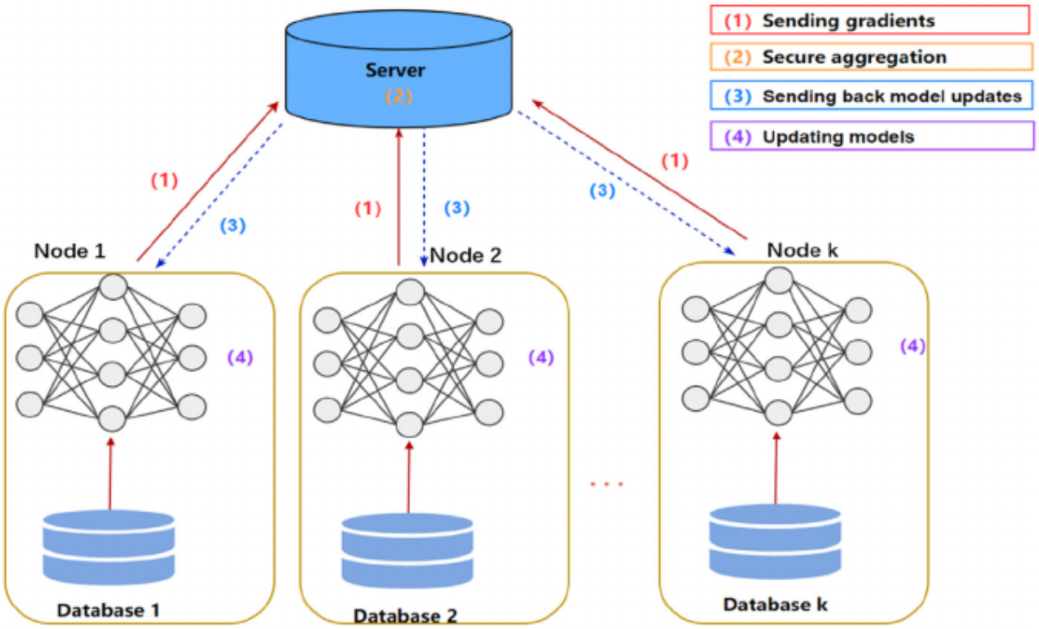


Figure 1.1: Overview of a typical distributed (federated) training round. Nodes compute local gradients on their private data (1), send them to the server for secure aggregation (2), receive the updated global model (3), and apply the new parameters locally (4).

optimization adds a coordination operation to this local computation.

2.4 Advantages and Challenges

DML can increase available computation, reduce dependence on a single machine, and allow data to remain near the devices that generate it. Its challenges include synchronization overhead, network congestion, heterogeneous devices, failures, non-identically distributed data, and the possibility that communication becomes more expensive than local computation. These challenges motivate federated and decentralized variants that treat communication as an explicit design dimension.

3 Federated Learning

3.1 Definition and Principles

Federated Learning (FL) is a distributed learning setting in which participating devices train a shared model while keeping their raw training data local. The defining workflow is that a coordinator sends model parameters or instructions to selected clients, clients perform local computation, and only model-related information is returned for aggregation [4], [5]. Keeping raw data local does not eliminate all privacy or security risks, but it changes the data-movement pattern and makes communication of updates central to system design.

Clients are typically weighted by the size of their local datasets, so that clients with more examples contribute more strongly to the global objective.

To clarify the fundamental difference between classical centralized learning and the federated paradigm, Figure 1.2 contrasts the two workflows. In the centralized setting, raw data must be collected at a single location before training can begin. In contrast, federated learning keeps the data on the client devices and exchanges only model-related information.

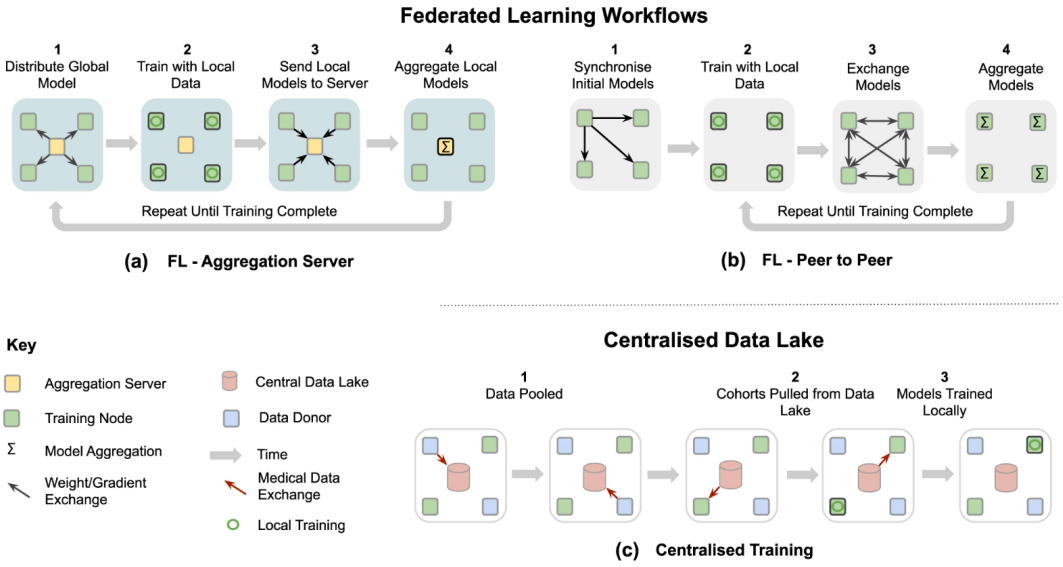


Figure 1.2: Comparison of centralized learning and federated learning workflows. In centralized learning, raw data is sent to a central server, whereas in federated learning only model updates are exchanged while data remains local.

This separation of data and computation is the defining characteristic that makes communication efficiency a central design concern in federated systems.

3.2 Federated Learning Workflow

At each round, the server selects a client set. It sends the current model to each selected client. Each client initializes from that model and performs a number of local updates. It then returns an update or local model. The server aggregates the received values and obtains the next global model.

This workflow separates local computation from global coordination. It also creates an uplink and downlink communication burden: model parameters travel to clients, and client updates travel back to the coordinator. The communication-efficient FL literature therefore studies update size, client participation, local computation, and the number of communication rounds together [5].

3.3 Federated Averaging

Federated Averaging (FedAvg) aggregates locally trained client models. Participating clients are weighted by their local sample counts. When every client performs one local step, FedAvg resembles distributed gradient aggregation; when clients perform several local steps, communication frequency decreases but the local models can drift in different directions [4], [6].

3.4 Challenges of Federated Learning

FL must handle statistical heterogeneity, device and network heterogeneity, partial participation, client availability, and communication constraints. Statistical heterogeneity means that the local objectives need not have the same minimizer. System heterogeneity means that clients can differ in computation time, memory, bandwidth, and energy. These factors affect both the quality of the aggregated update and the time required to complete a round.

4 Communication in Federated Learning

In federated learning, clients train models on their private local data and exchange only model updates with a central server. Figure 1.3 illustrates this communication process.

4.1 Communication Model

Communication volume is defined as the total number of bits transmitted between clients and the coordinator over all rounds. It depends on both message size and the number of transmissions. The definition can be specialized to bytes, packets, or energy if the unit is changed consistently.

4.2 Rounds and Communication Frequency

A communication round is a coordination interval in which a defined set of messages is exchanged and a new model state is produced. Communication frequency can be represented as the number of exchanges per unit time or, in local-update methods, as the number of local computation steps between two exchanges. Increasing the number of local steps between communications generally decreases the number of synchronization events for a fixed amount of local work; it does not necessarily decrease the size of an individual update.

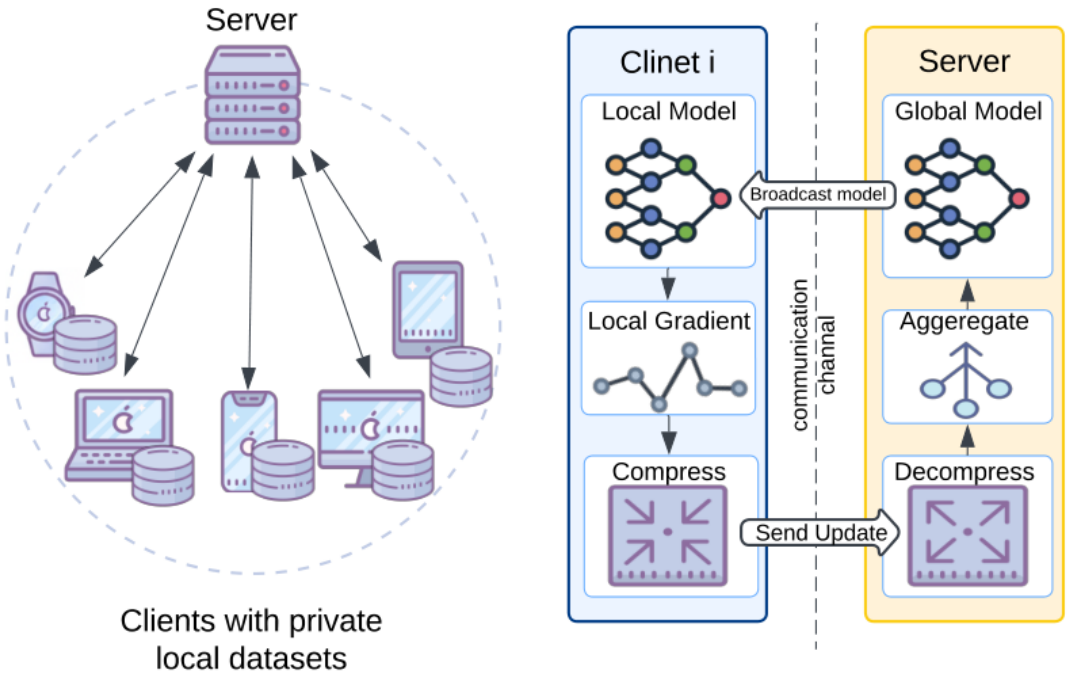


Figure 1.3: Federated learning communication architecture. Left: clients with private local datasets communicate with a central server. Right: one communication round, including model broadcast, local gradient computation, optional compression, and aggregation.

4.3 Volume, Message Size, and Message Count

Communication volume is the sum of the sizes of all transmitted messages. Message size is the amount of information in one transmission, whereas message count is the number of transmission events. A method can reduce volume by reducing average message size, by reducing the number of messages, or by reducing both.

4.4 Latency and Bandwidth

Bandwidth is a rate, usually expressed in bits per second. Latency is elapsed time. Under a simple link model, transmission time depends on message size divided by bandwidth plus fixed delays (propagation, queuing, processing, and protocol overhead). Lower latency does not automatically imply lower communication volume: a system may transmit the same number of bits faster, or transmit fewer bits over a slower link. The two quantities must be measured separately.

4.5 Communication–Computation Trade-off

Local computation can be increased to reduce synchronization frequency, while compression can reduce the size of each transmitted update. These choices affect accuracy, convergence, energy, and execution time. Communication-efficient FL therefore treats networking and optimization as coupled components rather than assuming that a reduction in one metric improves every other metric [5].

5 Decentralized Federated Learning

5.1 Motivation and Definition

Decentralized Federated Learning (DFL) removes the requirement that every update pass through one central aggregator. Nodes exchange model information over a peer-to-peer overlay and perform local aggregation using information received from neighbors. The DFL setting combines the local-data principle of FL with decentralized coordination and consensus mechanisms [7].

The absence of a mandatory central coordinator can reduce dependence on one server and can allow communication paths to follow the network topology. It also transfers coordination responsibilities to the nodes and the communication graph.

5.2 Centralized versus Decentralized Coordination

In centralized FL, communication is organized around a server. In DFL, each node may send to and receive from a subset of neighboring nodes. Centralized aggregation directly forms one global update, whereas decentralized aggregation progressively mixes information across the graph. Therefore, a decentralized system may require multiple hops or repeated exchanges before information from one node influences distant nodes.

5.3 Decentralized Aggregation

Each node aggregates models received from its neighbors (including possibly itself) according to a mixing matrix, then applies a local stochastic-gradient step. The mixing

weights are zero for non-neighbors. Convergence depends on graph connectivity, update timing, and the optimization assumptions [1], [2], [3].

5.4 Communication Challenges in DFL

DFL introduces neighbor selection, multi-hop information propagation, duplicate transmissions, topology changes, and possible disagreement among local models. The communication burden is no longer only the size of an update; it also depends on the number of active edges and the number of exchanges required for network-wide information mixing. Unreliable links can further create missing, delayed, or stale information [8].

6 Peer-to-Peer Communication

6.1 P2P Definition and Characteristics

A peer-to-peer network allows participating nodes to communicate directly without requiring all traffic to pass through a single coordinator. A peer can both originate information and relay or aggregate information received from other peers. In DFL, this property allows model updates to be exchanged along overlay links.

6.2 P2P Topologies

A topology specifies which node pairs can communicate. A complete graph connects every pair, but it requires a quadratic number of possible links. A sparse graph reduces the number of possible links but can increase path length and mixing time. Common abstract topologies include rings, grids, trees, random graphs, and expander-like graphs. The topology affects degree, connectivity, robustness, and communication volume.

The choice of communication topology has a direct impact on both the volume of exchanged messages and the speed of information mixing. Figure 1.4 illustrates the most common abstract topologies considered in the decentralized federated learning literature.

A dense topology such as a full mesh accelerates consensus at the cost of higher communication volume, while sparse structures (ring or scale-free graphs) reduce the number of messages but may increase mixing time.

6.3 Neighbor-Based Communication

If a node communicates with several neighbors and sends an update separately to each, the outgoing volume scales with the product of degree and message size. Broadcast support, multicast, or in-network aggregation can change this accounting. Consequently, node degree is a communication variable, not only a graph-theoretic property.

6.4 Gossip Communication

Gossip algorithms repeatedly exchange information between nodes or small neighbor subsets. In a pair-wise gossip step, two communicating peers mix their states with a mixing coefficient. Repeated exchanges can move the network toward consensus. The convergence speed depends on the interaction pattern and the mixing matrix; randomized gossip

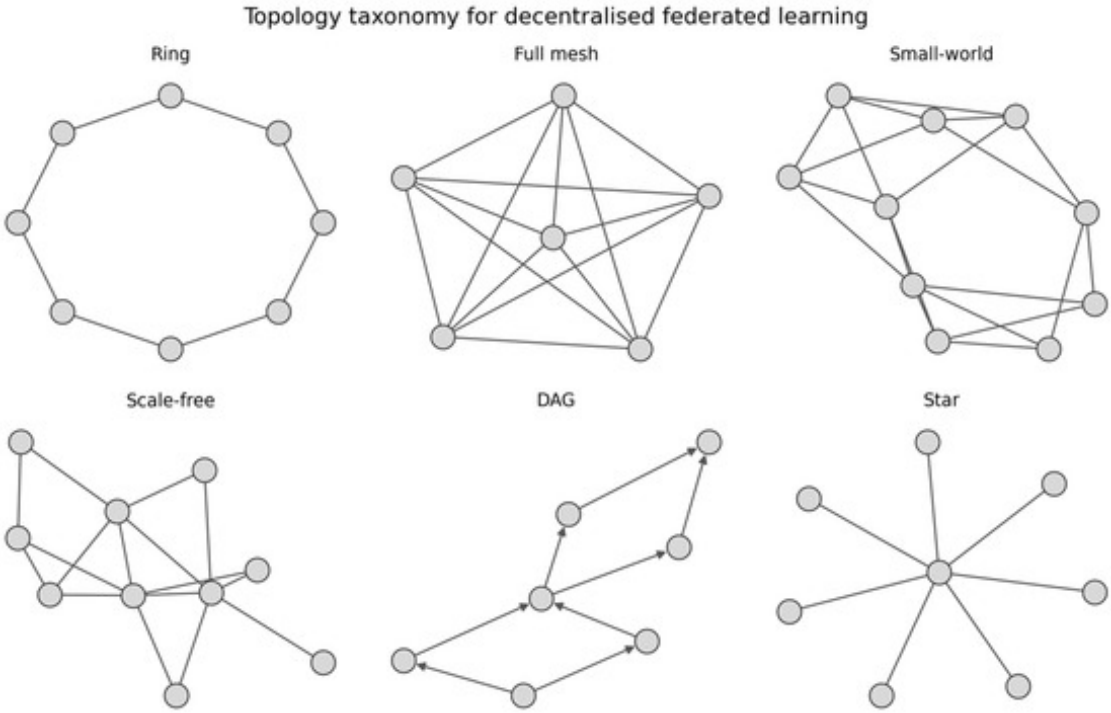


Figure 1.4: Common peer-to-peer communication topologies used in decentralized federated learning: ring, full mesh, small-world, scale-free, directed acyclic graph (DAG), and star.

provides a classical analysis of this relationship [9]. Pair-wise gossip has also been studied for decentralized federated learning in large networks [10].

7 Asynchronous Federated Learning

7.1 Synchronous Federated Learning

In synchronous FL, a coordination step waits for a prescribed set of client or neighbor messages before producing the next state. The barrier makes the aggregation point clear, but the slowest participating node can delay progress. The duration of a synchronous round is approximately determined by local computation time plus communication time of the slowest required participant.

7.2 Asynchronous Federated Learning

In asynchronous FL, a node can compute and communicate without waiting for all other nodes to finish the same round. Received updates are incorporated when they arrive. Asynchronous decentralized optimization can improve resource utilization in heterogeneous systems and avoid a global waiting barrier [11].

7.3 Synchronous versus Asynchronous Operation

Synchronous operation provides an easier-to-define iteration boundary and can simplify analysis. Asynchronous operation can reduce idle time and wall-clock waiting, but it

introduces update ordering, delayed information, and stale models. Importantly, asynchrony changes timing and coordination; it does not inherently reduce the number of bits transmitted. A reduction in communication volume requires a separate mechanism such as fewer exchanges, smaller messages, fewer neighbors, or selective transmission.

The distinction between synchronous and asynchronous operation is particularly important in heterogeneous networks. Figure 1.5 visualizes the two modes of coordination.

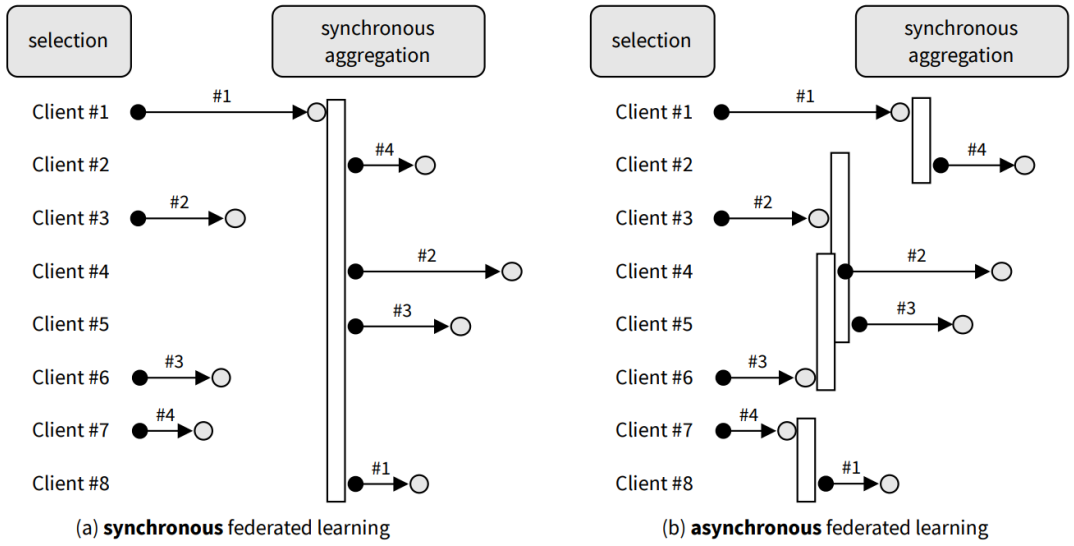


Figure 1.5: Comparison of synchronous and asynchronous federated learning. In the synchronous mode a global barrier waits for all selected participants, while in the asynchronous mode updates are incorporated as soon as they arrive.

While synchronous aggregation simplifies analysis and guarantees that all updates refer to the same model version, asynchronous execution can significantly reduce idle time when devices differ in computation speed or network latency.

7.4 Stale Updates and Asynchronous Decentralization

An update generated at an earlier logical time may be applied later; the difference is its staleness. The update may be computed from an older model and may therefore be less aligned with the receiving node's current state. A practical asynchronous method must balance reduced waiting against the error introduced by stale information. In the analysis used in this thesis, delay, topology, update frequency, and local objective differences are treated as separate system parameters rather than as a single communication metric.

8 Graph Theory and DAG-Based Systems

8.1 Graph Fundamentals

A graph consists of a set of vertices and a set of edges. In a communication graph, vertices represent nodes and edges represent permitted communication relationships. The degree of a vertex is the number of incident edges. A path is a sequence of connected vertices, and the path length counts its edges.

8.2 Directed Graphs

A directed graph contains ordered pairs, meaning that communication or dependency is directed from one vertex to another. Directed graphs can represent asymmetric links, one-way update propagation, or validation dependencies. In a directed communication graph, in-degree and out-degree may differ.

8.3 Directed Acyclic Graphs

A directed acyclic graph (DAG) is a directed graph with no directed cycles, and therefore admits a topological ordering of its vertices. In distributed learning systems, vertices typically represent update events or model states, while edges encode dependency relations.

A DAG describes organization and dependency; it may support parallel readiness of independent vertices, but the communication volume still depends on how many vertices are propagated, how large their payloads are, and how many peers receive them. Theoretical DAG modeling is related to broader work on learning directed acyclic structures [12].

8.4 DAG Coordination

A vertex is locally ready when the required information associated with its parent set is available. Independent vertices can be processed concurrently when their dependency sets do not overlap. Coordination nevertheless requires dissemination of identifiers, payloads, acknowledgements, or validation information. A DAG can therefore improve concurrency or traceability without automatically reducing message size, message count, or total communication volume.

9 Communication Optimization Fundamentals

9.1 Communication Efficiency

Communication efficiency is the ability to reach a target model quality with limited communication resources. It is not a single scalar property. A method can be efficient in transmitted bits but inefficient in wall-clock time, or efficient in latency but expensive in total messages.

9.2 Fundamental Metrics

The main metrics are:

- **Communication volume:** the total transmitted bits or bytes.
- **Message size:** the payload size of one message.
- **Message count:** the number of transmission events.
- **Communication frequency:** the number of transmission events per unit time or per local computation interval.
- **Latency:** elapsed time from message generation to reception or usable processing.
- **Bandwidth:** the available transmission rate.

- **Convergence time:** wall-clock time needed to reach a target quality or stopping criterion.
- **Communication rounds:** the number of logical coordination intervals.

In a decentralized setting, volume further depends on the set of active edges and the size of messages sent along those edges at each time step. Topology, neighbor degree, frequency, and compression must therefore be analyzed together.

9.3 Metric Relationships

Message size and message count are different dimensions. Communication frequency and convergence time are also different: reducing exchanges may reduce volume while increasing the time or number of local updates needed to reach the target accuracy. Likewise, asynchronous operation may reduce idle waiting while leaving volume unchanged. These distinctions prevent an optimization claim from silently changing from "fewer bits" to "lower latency" or from "fewer rounds" to "faster convergence."

10 Model and Update Compression

A variety of complementary techniques have been proposed to reduce the size or the frequency of model updates. Figure 1.6 provides a high-level taxonomy of the main families of communication-efficient methods.

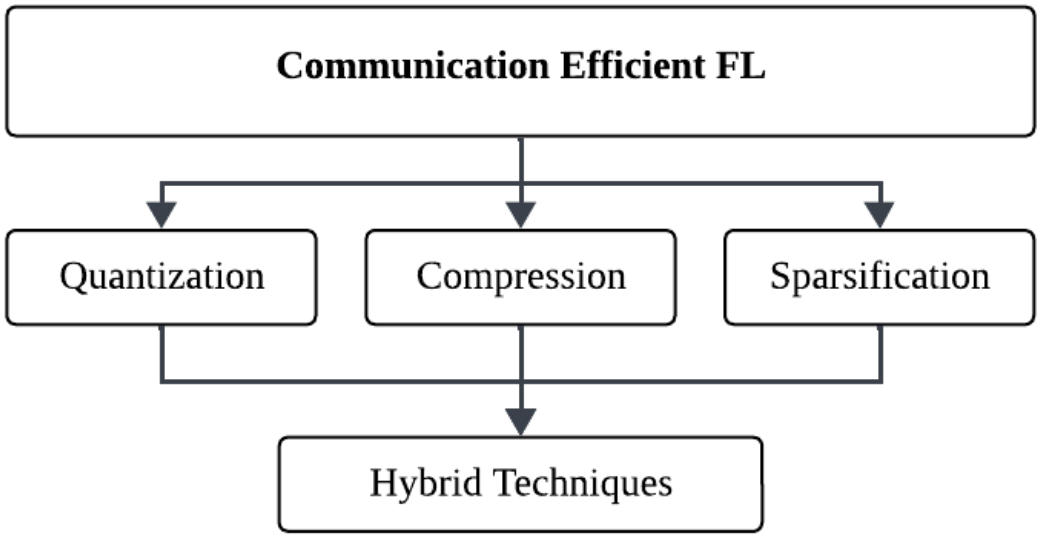


Figure 1.6: Taxonomy of the main communication-efficient techniques in federated learning: quantization, sparsification, and hybrid compression methods.

Quantization and sparsification primarily target message size, whereas selective transmission and hybrid schemes can also reduce the number of messages. The remainder of this section examines each of these approaches in more detail.

10.1 Quantization

Quantization maps a high-precision value to one of a finite set of representable values. Encoding each coordinate with fewer bits reduces the payload size, but introduces representation error. QSGD provides a representative analysis of quantized gradient communication [13], and quantized decentralized FL studies combine this idea with decentralized mixing [14].

10.2 Sparsification

Sparsification transmits only a subset of coordinates. The payload must include both the selected values and information identifying which coordinates were chosen. Sparsification reduces payload size only when the index and metadata costs do not cancel the savings. Error-feedback or memory mechanisms can preserve information omitted in one transmission; sparsified SGD is a representative example [15].

10.3 Pruning and Low-Rank Representation

Pruning removes parameters or update components considered unimportant. Structured pruning can remove complete filters, neurons, or blocks, which can be easier to encode than arbitrary individual coordinates. Low-rank representation approximates an update by factors of lower rank. Both approaches create an approximation–size trade-off and can add computation for selection, factorization, or reconstruction.

10.4 Selective Transmission

Selective transmission sends an update only when a condition is satisfied, for example when its magnitude exceeds a threshold, when a node has new information, or when a neighbor is selected. Selective transmission can reduce message count or volume, but excessive suppression may slow information propagation or increase disagreement among nodes.

10.5 Compression–Accuracy Trade-off

A compression operator introduces distortion. The exact convergence effect depends on the algorithm and assumptions. Arbitrary communication compression has been analyzed in decentralized learning, including the interaction between compression error and consensus [16].

11 Convergence and Communication–Performance Trade-offs

11.1 Consensus and Optimization

DFL has two related objectives: optimization, which seeks a low value of the global objective, and consensus, which seeks agreement among local models. A system may have low local disagreement but poor task accuracy, or good task accuracy while local models

remain different. Both dimensions should therefore be measured separately. Decentralized optimization analyses relate convergence to local computation, graph mixing, and communication assumptions [1], [2], [3].

11.2 Mixing and Convergence

For a mixing matrix, consensus behavior is affected by its stochasticity, graph connectivity, and eigenvalue structure. When appropriate assumptions hold, repeated mixing reduces disagreement. The spectral gap is commonly used as a topology-related indicator. This definition is a theoretical metric, not a guarantee that a larger gap alone improves the full learning task, because optimization noise and data heterogeneity also matter [3], [9].

11.3 Communication and Convergence Trade-offs

Increasing local computation between communications can reduce message count and volume per unit of training progress, but it can increase local drift. Increasing compression can reduce message size, but it can increase update distortion. Reducing the neighbor set can reduce per-round volume, but it can slow information mixing. Asynchrony can reduce waiting and improve resource utilization, but stale updates can complicate convergence. These are distinct trade-offs and should not be collapsed into a single claim that a method is simply "more efficient." Local SGD provides a formal example of the relationship between local work, communication, and convergence [6].

A useful evaluation therefore reports at least: target accuracy, total communicated bits, number of messages, average message size, communication rounds, wall-clock convergence time, and final model disagreement. Reporting only accuracy or only latency cannot establish communication efficiency.

12 Conclusion

This chapter established the theoretical vocabulary for the thesis. Distributed machine learning distributes computation and optimization across nodes; federated learning keeps raw data local while exchanging model-related information; and decentralized federated learning replaces mandatory server aggregation with neighbor-based coordination. Peer-to-peer and gossip mechanisms determine how information propagates, while synchronous and asynchronous operation determine when nodes exchange information.

The chapter also formalized the distinction between communication volume, message size, message count, communication frequency, latency, bandwidth, rounds, and convergence time. Compression mechanisms—including quantization, sparsification, pruning, low-rank representation, and selective transmission—mainly target message size or message count, whereas asynchronous execution mainly targets waiting and timing. DAGs describe dependency and coordination structure but do not by themselves guarantee lower communication volume. These foundations support the State of the Art in Chapter 2, the architectural description in Chapter 3, the implementation in Chapter 4, and the experimental metrics in Chapter 5.

Chapter 2

State of the Art

1 Introduction

The rapid proliferation of edge devices and the growing demand for privacy-preserving machine learning have established Federated Learning (FL) as a cornerstone of distributed artificial intelligence. Since FedAvg [4], FL has evolved from a simple client-server averaging scheme into a broad family of collaborative learning paradigms. Centralized FL, however, suffers from communication bottlenecks at the server, single points of failure, and scalability limits at scale.

Decentralized Federated Learning (DFL) removes the central coordinator in favor of peer-to-peer (P2P) interactions [7]. This removes the single point of failure but introduces new communication complexities tied to topology, gossip dissemination, and asynchronous coordination: without a global view, no entity can regulate the volume or timing of exchanges, so communication efficiency becomes a systemic property coupling topology, reliability, and compression together.

This chapter reviews the state of the art in communication optimization for DFL, guided by the question:

> *How can communication efficiency be improved in a decentralized federated learning system while preserving model performance, convergence, scalability, and decentralized operation?*

Section 2 characterizes FL's communication challenges. Section 3 reviews communication-efficiency techniques from centralized FL. Sections 4 and 5 cover the shift to decentralized topologies and the communication mechanisms specific to DFL. Section 6 surveys DFL-specific optimization techniques, and Section 7 focuses on DAG-based coordination. Section 8 gives a detailed, paper-by-paper review of the most representative recent contributions, followed by a synthesis (9) and a comparative table (10). Sections 11–13 discuss limitations, the resulting research gap, and the positioning of this thesis.

2 Communication Challenges in Federated Learning

Communication is the primary bottleneck in FL, especially when deploying large models on resource-constrained edge devices [14], [17]. The communication volume is

$$V_{\text{comm}} = R \times N_{\text{active}} \times S_{\text{update}},$$

where R is the number of rounds, N_{active} the active nodes per round, and S_{update} the average update size. Optimizing this volume requires acting on frequency, magnitude, participation, and, in DFL, topology.

- **Update size and frequency:** models with millions of parameters exceed the bandwidth of many edge devices [14], [18]; frequent synchronization helps per-round convergence but strains total bytes transferred.
- **Bandwidth and latency:** wireless links exhibit asymmetric, variable-latency traffic [19], hindering timely dissemination.
- **Heterogeneity and stragglers:** slower nodes bottleneck synchronous aggregation [20], [21].
- **Unreliable connections:** dropped or partially received updates can bias aggregation toward well-connected nodes if not corrected [8], [22].
- **Redundancy and scalability:** gossip dissemination causes duplicate transmissions that become prohibitive at scale [23], [24], motivating sub-quadratic schemes [25].

These factors motivate five optimization axes used throughout this chapter: fewer transmissions, smaller updates, fewer participants per round, better topology, and more informative synchronization.

3 Communication-Efficient Centralized Federated Learning

3.1 Reducing Rounds and Selecting Clients

FedAvg [4] showed that multiple local epochs before transmission reduce communication rounds; Stich formalized that Local SGD converges comparably to synchronous SGD with far less communication [6]. Konecny et al. proposed structured/sketched updates and lossy compression as complementary strategies [5]. Excessive local training, however, causes drift under non-IID data [18], [26]. To manage bandwidth, servers often select only a fraction of clients per round, from random to importance- or resource-aware schemes [27], [28].

3.2 Gradient and Model Compression

The general pipeline is

$$\text{Original Update} \rightarrow \text{Compression} \rightarrow \text{Transmission} \rightarrow \text{Reconstruction}.$$

Quantization maps weights to lower-bit representations [14], [29]; QSGD formalized the quantization–variance trade-off [13], and Quantization-Aware Training integrates quantization error into training for robustness [30].

Sparsification transmits only the largest-magnitude gradients [24], [31]. Stich et al. showed that error-feedback memory largely compensates for discarded information [15]. FedSparQ extends this to a fully adaptive setting, jointly tuning the sparsification ratio

and quantization level per round via error feedback, to remain robust under non-IID data and bandwidth fluctuations [31].

Figure 2.1 shows this pipeline: sparsify with an adaptive top-k, quantize the survivors, and feed the discarded residual back into the next round rather than dropping it.

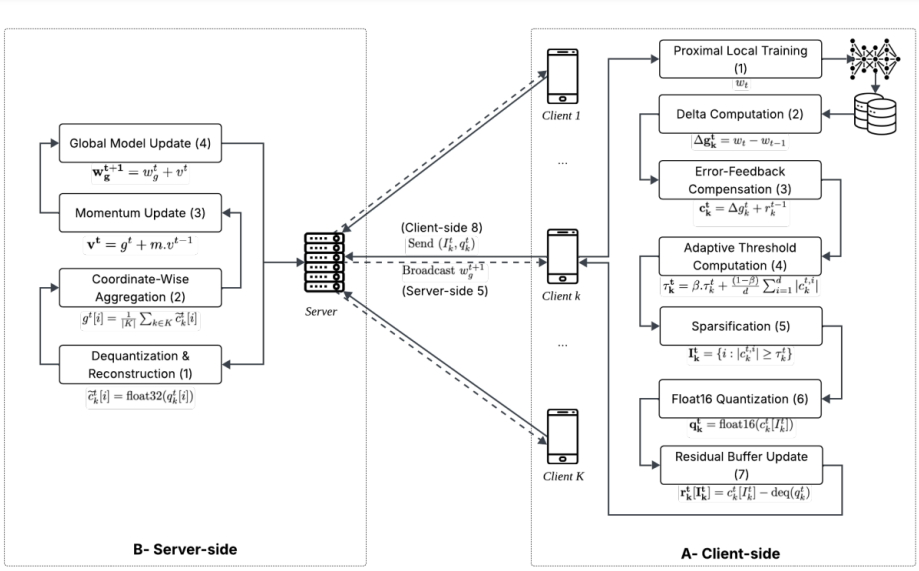


Figure 2.1: FedSparQ's per-round pipeline: adaptive top-k sparsification, low-bit quantization, and an error-feedback loop that re-injects the compression residual into the next update [31].

The residual-feedback loop is what keeps a heavily compressed update close, in aggregate, to its uncompressed counterpart over several rounds — a property this thesis' own compression insertion (Section 13) must also preserve to remain compatible with cryptographic verification.

Pruning and low-rank reduction remove redundant parameters or factorize gradients into lower-dimensional forms [32]. Kritsiolis and Kotropoulos combine low-rank approximation with quantization of the factors, showing the two compress multiplicatively rather than redundantly [33].

Knowledge distillation lets nodes exchange compact class-level logits or prototypes instead of full models [34], enabling heterogeneous architectures at the cost of dependence on distillation data quality.

4 Transition to Decentralized Federated Learning

Removing the central server addresses its bottleneck and single point of failure [7], [10], building on distributed optimization foundations [2], [9]: nodes exchanging information only with neighbors can still reach global consensus if the graph is connected. In a P2P system, each node communicates only with immediate neighbors, and Lian et al. showed decentralized parallel SGD can match centralized convergence while removing the server bottleneck, given a well-connected network [3], [11]. This shift requires new peer-discovery and propagation mechanisms, and introduces propagation latency, redundancy, and staleness under asynchrony [19], [35].

5 Communication in Decentralized Federated Learning

5.1 Gossip and Decentralized Optimization

Gossip protocols disseminate updates by neighbor-to-neighbor exchange [23], [24]. DIGEST combines frequent local updates with sparse, triggered gossip, bounding local drift between exchanges [23]. Naive gossip is inefficient in large-diameter networks; adaptive matrices and arbitrary-compression analyses improve this [16]. Decentralized optimization (D-SGD) achieves consensus via local averaging, with efficiency governed by the spectral gap of the mixing matrix [26]; dual averaging offers an alternative primitive for lossy, asynchronous settings [1].

5.2 Asynchrony, Peer Selection, and Topology

Asynchronous DFL removes the global clock but introduces stale updates [20], [36], addressed via delay-tolerant and reception-aware aggregation [22], [35]. Kaluarachchi and Buyya prove that classical gossip introduces selection bias proportional to link-loss rate, correcting it with Inverse Probability Weighting and Age-of-Information decay [22].

Peer selection targets the communication-convergence trade-off via data similarity or proximity [27], [28]; OCD-FL frames it as an opportunistic decision balancing knowledge gain against energy, reporting 30–80% energy savings [37]. Dynamic selection further adapts topology as nodes join or leave [38].

Topology strongly influences efficiency: fully connected graphs offer fast consensus but do not scale, while Ring, Grid, and Erdős-Rényi graphs trade connectivity for cost [39]. MSTs and 2D-Rings minimize dissemination time [40], [41], and AIRPLAN recasts topology selection as a query-optimization problem over a cost model built from privacy-preserving sketches [42].

6 Communication Optimization Techniques for Decentralized FL

Round reduction combines multiple local updates with fewer or grouped gossip steps [43], [44]. Volume reduction restructures aggregation itself: MAR-FL's iterative group-based aggregation reduces complexity from $\mathcal{O}(N^2)$ to $\mathcal{O}(N \log N)$ while remaining resilient to churn [25]. Compression must integrate with the decentralized workflow: GossipFL sparsifies with an adaptive gossip matrix [24], while Quantized Rank Reduction shows centralized compression pipelines transfer naturally to pairwise P2P exchanges [33]. Adaptive, event-triggered communication transmits only when update magnitude exceeds a threshold [45], [46]; SPARQ-SGD combines this with quantization and sparsification while retaining the convergence rate of full-communication decentralized SGD [45]. Communication-aware peer selection, e.g. by SNR, further concentrates resources where they matter [42], and asynchronous frameworks such as DRACO enable continuous, non-blocking exchange across heterogeneous schedules [20].

7 DAG-Based Federated Learning and Communication

Directed Acyclic Graphs (DAGs) offer a decentralized alternative to blockchains or flat gossip for coordinating updates: each update is a vertex referencing prior updates (tips), forming a multi-threaded training process [47], [48].

Figure 2.2 illustrates this principle: rather than a single linear chain, each new transaction attaches to one or more existing tips, letting several branches grow and validate concurrently.

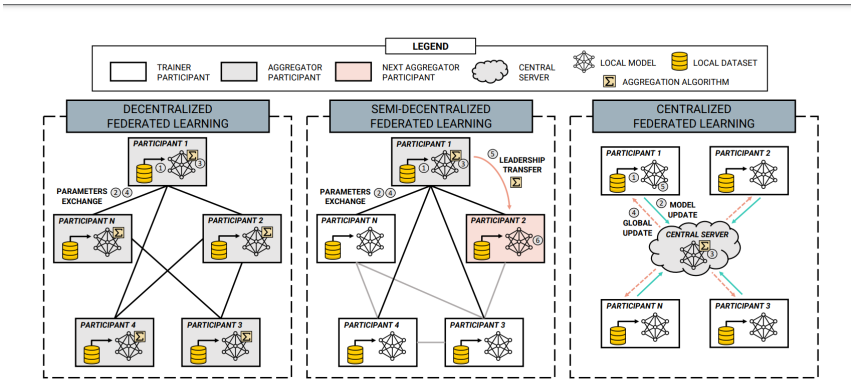


Figure 2.2: Generic tip structure of a DAG-based FL system: each new update references existing tips, forming a growing graph instead of a single chain.

This absence of a canonical chain is what lets a DAG absorb concurrent, asynchronous updates without a global sequencing bottleneck; the trade-off is that the *tip-selection rule* becomes the main lever for convergence quality and redundant propagation. Nodes select and validate tips before attaching new updates, naturally handling stragglers without a server [46]. Accuracy-based and adaptive tip selection improve security and personalization [49], [50], but poor selection causes latency and redundancy. Whether DAGs primarily improve efficiency or mainly provide asynchronous coordination remains debated [46], [47]: most DAG-FL systems report communication volume, if at all, only as a secondary outcome — a point central to the research gap in Section 12.

8 Detailed Literature Review of Representative Papers

8.1 ProFe

Sánchez et al.'s ProFe combines knowledge distillation, prototype learning, and 16-bit quantization [34]: large teacher models distill into compact students for transmission, while shared class prototypes compensate for classes under-represented at other nodes.

Figure 2.3 shows this per-node pipeline: the teacher trains on raw data, distills into a student, and prototypes are computed and shared separately from the model channel.

Only the model channel need be architecture-compatible across devices, enabling heterogeneous students. On MNIST/CIFAR-10/CIFAR-100 with 20 fully connected nodes, ProFe reports 40–50% communication reduction versus FedAvg, at the cost of distillation overhead and an evaluation topology that does not reflect sparser real deployments.

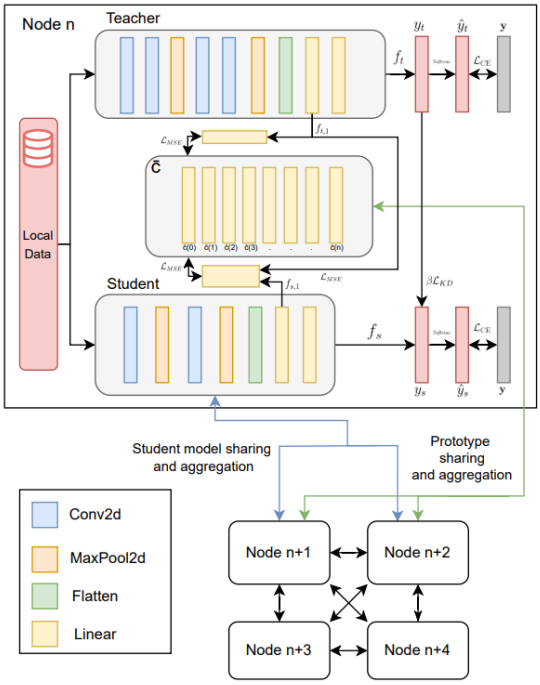


Figure 2.3: Per-node architecture of ProFe: teacher-to-student distillation plus a separate prototype channel for unseen-class recognition [34].

8.2 GossipFL

GossipFL adapts sparsification and the gossip matrix jointly, per-node bandwidth and compute estimates [24], cutting traffic and time by over an order of magnitude in some settings, though periodic profile re-estimation adds unaccounted control-plane overhead.

8.3 MIMO Over-the-Air Computation

Zhai et al. exploit wireless superposition for simultaneous transmission and aggregation via MIMO AirComp, jointly optimizing beamformers and the mixing matrix [51]. This reduces latency but channel noise corrupts the aggregate directly, and digital error-feedback compression cannot straightforwardly apply.

8.4 MAR-FL

MAR-FL's iterative group-based aggregation reduces P2P complexity from $\mathcal{O}(N^2)$ to $\mathcal{O}(N \log N)$ and tolerates churn [25], though the interaction between group formation and non-IID data across groups remains unexplored.

8.5 BCFL

BCFL couples Top-k compression with a dynamically adjusted compression rate under a fixed training-time budget in a blockchain-based FL setting [52], but per-block consensus latency remains a cost that DAG-based coordination (Section 7) is designed to avoid.

8.6 Asynchronous Approaches: DRACO and FedDual

DRACO uses continuous row-stochastic mixing matrices, bounding messages received per unit time [20]; FedDual performs asynchronous pair-wise gossip with convergence comparable to centralized training [10], at the cost of a slower mixing rate on poorly connected graphs.

8.7 Reception- and Age-Aware Aggregation

Kaluarachchi and Buyya prove classical gossip is biased toward well-connected nodes under lossy links, and correct this with Inverse Probability Weighting plus Age-of-Information decay, without a global clock [22]. This is the clearest treatment in the reviewed literature of reliability and volume as distinct, jointly required objectives.

8.8 PISCO

Wang and Chi's PISCO lets agents communicate with a central server or with neighbors in a probabilistic, semi-decentralized manner [53].

Figure 2.4 shows the protocol: dotted lines denote agent-to-server contact (probability p), solid lines denote agent-to-agent gossip used whenever the server is unavailable.

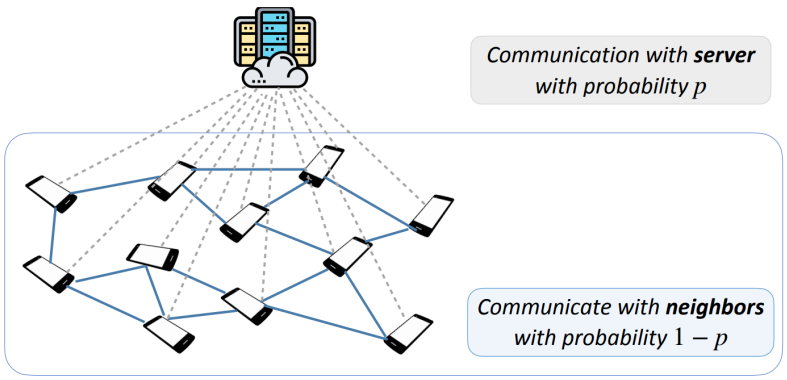


Figure 2.4: PISCO's semi-decentralized protocol: agents reach the server with probability p and otherwise gossip with neighbors [53].

Even limited server access substantially cuts agent-to-agent rounds, especially on poorly connected graphs, suggesting that a small, controlled amount of centralization can beat full decentralization in sparse networks — relevant to the shard-chief role used in this thesis' architecture.

8.9 AIRPLAN

Attaluri et al.'s AIRPLAN treats each Over-the-Air aggregation round as an approximate distributed query and topology selection as cost-based query optimization [42]: the communication graph is the execution plan, the mixing-matrix spectral gap is a cardinality estimate, and top-k sparsification is an approximate-query operator.

Figure 2.5 shows the overall system: clients, the OTA channel, and the topology advisor that selects the graph from privacy-preserving Count-Min Sketch statistics.

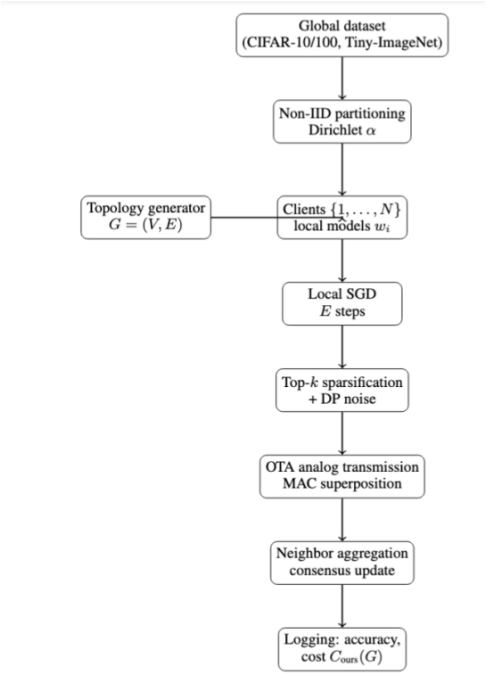


Figure 2.5: Overall architecture of AIRPLAN: clients, the OTA aggregation channel, and the topology advisor that selects the communication graph [42].

Figure 2.6 then shows the resulting accuracy-cost trade-off across topology families, positioning each on a Pareto-style frontier.

Across five topology families, three vision benchmarks, four client scales, and multiple SNR levels, AIRPLAN matches the oracle-optimal topology in 91.4% of workloads with under 1.8% overhead, directly coupling the *topology* and *size/frequency* axes of Section 9. It remains limited to OTA aggregation and untested on physical hardware.

8.10 Quantized Rank Reduction and OCD-FL

Kritsiolis and Kotropoulos project gradients onto a low-rank subspace before quantizing the coefficients [33], reducing bytes with minor accuracy loss but no P2P/gossip evaluation. Masmoudi and Jaafar's OCD-FL instead optimizes *which* links to use: each node weighs expected knowledge gain against energy cost per candidate neighbor, achieving accuracy on par with full gossip at 30–80% less energy [37] — complementary to compression, since it reduces the number of exchanges rather than their size.

8.11 Graph-Based Gossiping and DAG-AFL

Nguyen et al. schedule gossip via Minimum Spanning Trees and graph coloring to avoid collisions, cutting bandwidth and transfer time, though scheduling overhead grows with topology volatility [40].

DAG-AFL replaces PoW-style consensus with a DAG whose tip selection weighs freshness, reachability, and accuracy, plus lightweight trusted verification [48].

Figure 2.7 details this: a new update is scored on the three criteria, attached to selected tips, and later validated by peers before it can itself become a tip.

Complementary work on sharded DAGs uses change-point detection to adapt how many tips a new update should reference, improving concurrency and fairness without a

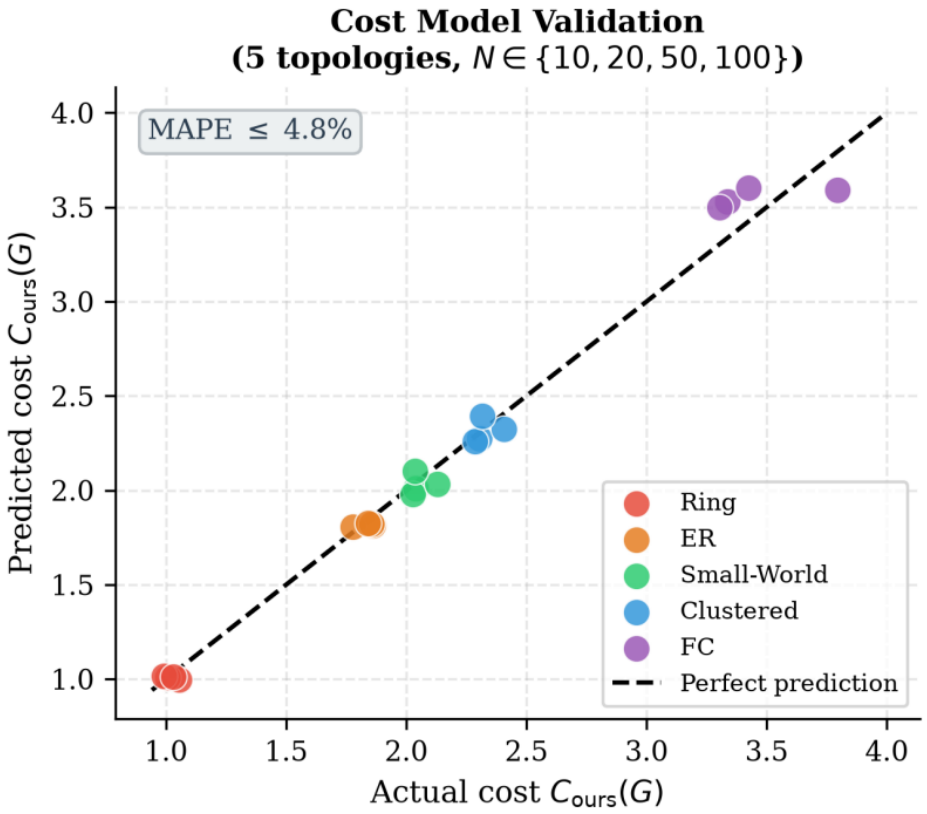


Figure 2.6: Ring, Grid, Erdős-Rényi, small-world, and clustered topologies on the accuracy-cost Pareto frontier [42]. Small-world/clustered graphs reach ≈90% of a fully connected graph's accuracy gain at ≈57% of its cost.

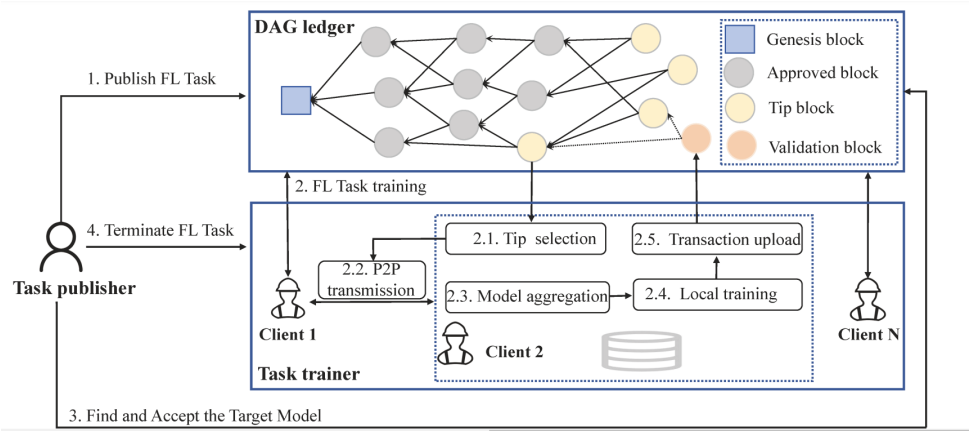


Figure 2.7: DAG-AFL's pipeline: freshness/reachability/accuracy scoring, attachment, and peer verification [48].

fixed tip count [54]. In both cases, communication-volume reduction remains a secondary consequence rather than the primary design objective.

9 Cross-Paper Critical Synthesis

Five axes recur across the literature: (1) **number of communications** (local rounds, event-triggering, group aggregation [25], [43], [46]); (2) **size** (quantization, sparsification, low-rank, distillation [24], [30], [31], [33], [34]); (3) **topology** (MSTs, small-world/clustered graphs, query-optimized selection [28], [40], [41], [42]); (4) **timing/reliability** (asynchrony, staleness, link-loss bias [10], [20], [22]); and (5) **usefulness** (peer/client selection, prototypes [27], [34], [37]).

No reviewed system optimizes more than two axes at once inside a verifiable, sharded architecture: compression papers (ProFe, GossipFL, QRR, FedSparQ) are evaluated on flat, often fully connected topologies, while topology-optimization papers (AIRPLAN, Nguyen et al., MAR-FL) treat payload size as fixed. The reliability axis (DFL-AA) and DAG coordination have so far evolved separately, even though both target asynchronous, unreliable, multi-hop propagation — combining them, as intended here, is unexplored.

10 Comparative Analysis of Existing Approaches

Table 2.1: Comparison of communication-efficient decentralized FL approaches

Approach	**Year**	**Main Technique**	**Target**	**Main Limitation**
ProFe	2024	KD + Prototypes	Bytes	Training overhead; FC-only evaluation
GossipFL	2023	Sparsification + Adaptive Gossip	Traffic / Time	Control-plane overhead
MIMO OA-DFL	2023	AirComp	Latency	Noise; no error feedback
Nguyen et al.	2025	MST + Graph Coloring	Bandwidth	Scheduling under churn
MAR-FL	2025	Group Aggregation	Scalability	Non-IID across groups
AIRPLAN	2026	Query-optimized topology	Total Cost	OTA-only; no hardware test
QRR	2025	Low-rank + Quantization	Bytes	No P2P evaluation
FedSparQ	2025	Adaptive Sparsify + Quantize	Bytes	Adaptive-ratio tuning
OCD-FL	2024	Opportunistic Peer Selection	Energy	No explicit compression
PISCO	2023	Probabilistic Semi-decentralized	Rounds	Residual server dependence
DRACO	2024	Row-stochastic Async SGD	Redundancy	Asynchrony management
DFL-AA	2026	Reception / Age-aware Aggregation	Bias / Stale	Online channel estimation
BCFL	2022	Top-k Compression	Traffic / Time	Blockchain latency
DAG-AFL	2025	DAG + Freshness / Accuracy Tips	Consensus	Limited volume analysis
DAG-Shard	2024	Adaptive Tip Selection	Concurrency	Volume not primary metric

11 Critical Analysis and Research Limitations

Despite this progress, several gaps persist. **Gossip redundancy** remains significant in dense networks [23], [24]. **Graph maintenance** for MSTs or 2D-Rings adds overhead under churn that is rarely counted [40], [55]. **Heterogeneity** is often idealized away, understating its effect on asynchronous convergence [20], [21]. The **compression-convergence trade-off** lacks tight theoretical limits for decentralized topologies [14], [56]. **Evaluation** rarely reports byte-level volume or energy [18], [57], and even when it does (ProFe, GossipFL, MAR-FL), figures usually come from in-process simulation rather than serialized payloads, understating real framing and verification overhead. Finally, most DAG-FL work ([46], [47], [48], [49], [54]) optimizes tip correctness, Byzantine resilience, or fairness, while compression/topology papers rarely include cryptographic verification of the transmitted update — a disjunction that motivates the research gap below.

12 Research Gap

The intersection of DFL, P2P communication, and extreme communication efficiency remains under-explored. Compression and topology optimization have been studied separately; their integrated deployment within a sharded, DAG-based, verifiable framework is largely unaddressed. Even the most recent contributions reviewed here — AIRPLAN's topology selection [42], MAR-FL's sub-quadratic aggregation [25], and DAG-Shard's adaptive tip selection [54] — each cover only one segment of the pipeline; none combines gradient-level compression with verifiable, sharded DAG coordination and real, serialized measurements. Most DAG-FL work prioritizes security and immutability over communication [58], [59], leaving open how DAG multi-threading can be combined with hierarchical sharding to limit propagation volume while preserving asynchrony and verifiability, and how QAT, Top-k, and low-rank compression [31], [33] interact with multi-hop DAG propagation and zero-knowledge verification of the compressed payload.

13 Positioning of the Master's Thesis

This thesis builds on the author's Graduation Project (PFE), *Decentralized DAG-Based Federated Learning System*, which introduced a three-layer Client–Shard–Global architecture coordinated via a DAG. The Master's research pivots from security and basic coordination toward communication optimization, enhancing the architecture through:

1. Top-k (magnitude) gradient sparsification with optional low-bit fixed-point quantization at the client-to-shard-chief boundary, engineered so that (a) zero-knowledge proofs and polynomial proxies still certify the original clipped update, (b) commitments bind to the compressed payload, and (c) the shard chief deterministically decompresses before reusing the existing robust aggregation pipeline.

2. A Communication Metrics Logger that records real serialized byte counts for original and compressed representations at the network boundary, producing reproducible compression ratios instead of architectural estimates — directly addressing the evaluation gap of Section 11.

3. An analytical communication model and pseudocode for compression/decompression, enabling empirical bounding of the compression–accuracy trade-off under non-IID, asynchronous conditions.

Unlike GossipFL (sparsification without shard chiefs or ZKP verification [24]), ProFe (distillation reliant on public data and heterogeneous setups [34]), AIRPLAN and MAR-FL (topology/ aggregation optimization without payload compression or verification [25], [42]), or DAG-AFL and DAG-Shard (DAG coordination without a communication-volume objective [48], [54]), this work inserts compression at a specific architectural boundary in a hierarchical, verifiable, DAG-coordinated system, and measures the result directly. The contribution is therefore: (i) a compatibility-preserving compression pattern for verifiable sharded DFL, (ii) a measurement strategy that makes savings verifiable rather than simulated, and (iii) an evaluation plan quantifying accuracy loss and convergence under realistic conditions.

14 Conclusion

DFL's state of the art has advanced along physical-layer (AirComp, AIRPLAN), algorithmic (MAR-FL, ProFe, FedSparQ, QRR), reliability (DFL-AA), and coordination (DAG-AFL, DAG-Shard) fronts. Yet, as Sections 9–12 show, no single system jointly addresses payload compression, DAG-based or topology-aware coordination, and verifiable, measured communication savings. This review establishes the baseline for such an integration; the next chapter presents the proposed communication-enhanced architecture.

Chapter 3

Proposed Communication-Enhanced Architecture

1 Introduction

This chapter addresses the communication bottleneck of a hierarchical, sharded, DAG-coordinated and zero-knowledge-verified decentralized federated learning system. While the system already provides strong integrity and Byzantine-resilience guarantees, every client still transmits a full-dimensional model update to its shard chief in each round. The objective is therefore to reduce communication volume without weakening the existing verification, commitment or robust aggregation mechanisms.

The chapter is organised as follows. Section 2 summarises the communication behaviour of the current system. Section 3 formalises a general communication model and states the requirements that any optimisation must satisfy. The remainder of the chapter presents the proposed Communication-Enhanced Architecture: a gradient compression layer inserted at the client-to-shard-chief boundary. Throughout, a clear distinction is maintained between components that already exist and those that constitute new contributions of this thesis.

2 Summary of the Existing Decentralized Federated Learning System

2.1 System Topology

The existing system is organized into three layers, as established in Chapter 2:

- **Client Layer**: a set of clients, partitioned across a fixed number of shards.
- **Shard Layer**: one Shard Chief per shard, elected via a score-based mechanism combining uptime, bandwidth, and compute capability, with a top-candidate pool and randomized selection among the leading candidates to support rotation.
- **Global Layer**: one Global Coordinator that aggregates shard-level models and broadcasts the updated global model.

This hierarchical topology is a structural property of the base system and is independent of the number of clients or shards used in any particular deployment or experiment.

The three-layer hierarchy ensures fault tolerance and enables decentralized governance, with each layer operating independently while coordinating through well-defined message boundaries.

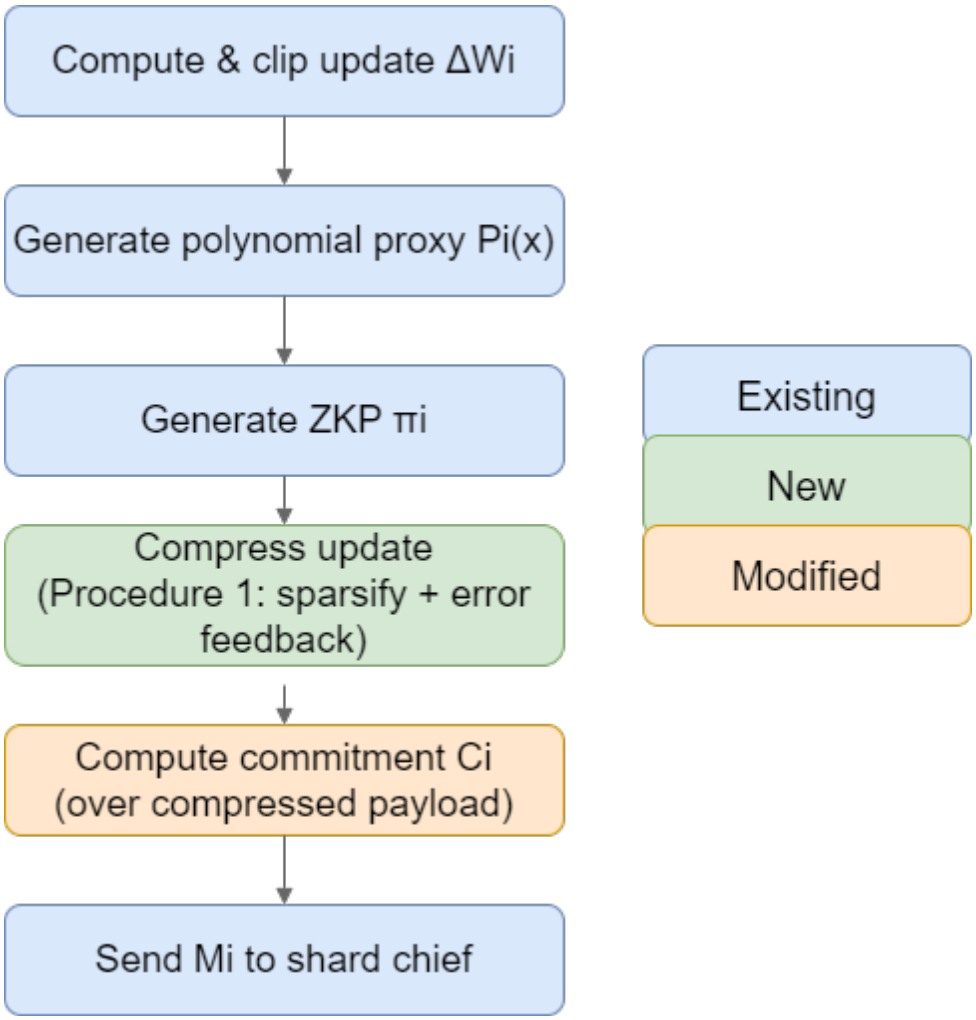


Figure 3.1: Client-side pipeline showing the processing stages from local training to update transmission. Each client follows a strict sequence of operations: gradient computation, clipping to norm bounds, polynomial proxy generation, zero-knowledge proof computation, optional compression, and commitment generation to ensure integrity of the transmitted payload.

2.2 Per-Round Communication Flow

The base system executes training in discrete rounds, following the flow:

$$\text{Global} \longrightarrow \text{Shards} \longrightarrow \text{Clients} \longrightarrow \text{Shards} \longrightarrow \text{Global} \tag{3.1}$$

Concretely, in each round, the following steps occur:

1. The Global Coordinator broadcasts the current global model to all clients, via the shard chiefs.

2. Each client performs local training and computes an update equal to the difference between its locally trained model and the received global model, clipped to a fixed norm bound.
3. Each client constructs a low-degree polynomial proxy summarizing its update, intended for cheap downstream verification rather than for aggregation.
4. Each client computes a cryptographic commitment to its update.
5. Each client generates a zero-knowledge proof attesting that its update was correctly computed and satisfies the norm bound.
6. Each client submits its update, together with the proof, the commitment, and the polynomial proxy, to its shard chief.
7. The shard chief verifies each proof and applies the existing multi-stage robust aggregation procedure (proof-based filtering, reputation-based filtering, cluster consensus, and reputation weighting) to produce a shard-level aggregate.
8. Each verified update is recorded in the per-shard DAG, referencing only a model hash and the commitment — not the raw update itself.
9. The shard chief forwards the shard-level aggregate to the Global Coordinator.
10. The Global Coordinator aggregates all shard-level models into the next global model and broadcasts it for the following round.

Figure 3.2 illustrates the cyclic communication pattern across all three layers, showing how updates flow from clients to shard chiefs and then to the global aggregator. The per-round communication flow reveals the critical bottleneck: the client-to-shard-chief hop, which involves the largest number of concurrent messages and carries the highest volume of model-specific data, making it the natural target for optimization.

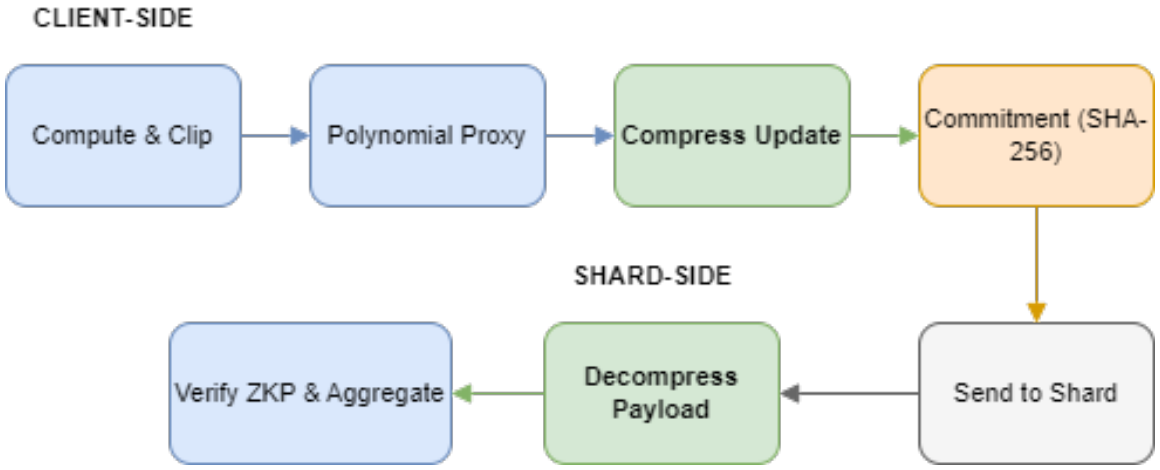


Figure 3.2: Per-round communication flow across the three-layer architecture. Updates originate at clients, pass through shard chiefs for verification and robust aggregation, and culminate at the global coordinator for model aggregation. This cyclic pattern defines the baseline communication profile; the proposed compression optimization targets the client-to-shard boundary where message count and volume are highest.

2.3 Implementation-Level Observations

Direct inspection of the implementation (the client module, the shard-chief module, the training-orchestration module, and the peer-to-peer networking module) yields the following facts, which are foundational to the communication model formalized in Section 3:

- In the client module, the update-creation routine produces the local update as a full-dimension, full-precision numerical array covering every parameter of the model. No compression, quantization, or sparsification is applied to this update prior to transmission.
- The polynomial proxy reduces the complexity of *verification* from a cost proportional to the model dimensionality to a cost proportional to a small, fixed polynomial degree. It is not, however, a substitute for the transmitted update: aggregation requires the true, full values of the update, so the complete vector must still be conveyed to the shard chief.
- In the shard-chief module, the round-processing routine accepts the client updates directly as in-memory numerical arrays.
- In the training-orchestration module, the routine responsible for executing a training round passes each client's update directly by object reference from the client-update collection step into the shard-chief's round-processing routine. No serialization (for example, converting the object to a byte stream) or network transmission (for example, a socket send) occurs along this path.
- The peer-to-peer networking module implements a generic task-dispatch interface, used for auxiliary tasks unrelated to model updates, and employing its own serialization scheme for that purpose. This module is not invoked anywhere in the client-update or shard-aggregation path.

Figure 3.3 provides a high-level overview of how the core system modules interact during a training round. The shard-chief pipeline, detailed in Figure 3.4, shows the verification and aggregation stages that operate on the full-dimensional updates received from clients.

Implication. The base system, as implemented, executes the federated round as an in-process simulation: updates are passed by reference within a single process rather than serialized and transmitted across a real network boundary. Communication cost in the base system is therefore not empirically observable as transmitted bytes; it must be introduced explicitly as part of the proposed architecture (Section 4) in order to be measured and optimized.

3 Communication Model and Requirements

3.1 Notation

All quantities below are expressed in terms of these symbols only, so that the model applies regardless of the specific number of clients, shards, or the specific model architecture used in a given deployment.

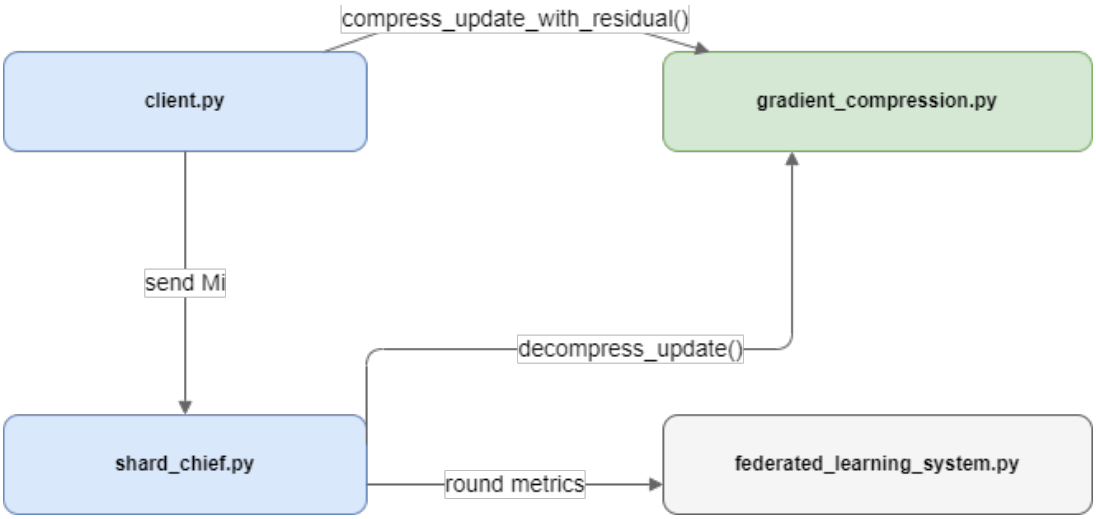


Figure 3.3: Module interaction diagram showing the information flow between client.py, gradient compression layers, shard_chief.py, and system-level orchestration. The compression module and decompression module sit at the client-to-shard boundary, enabling measurement of real communication statistics while leaving downstream verification and aggregation untouched.

Table 3.1: Notation used in the communication model.

Symbol	Meaning
N	Number of clients
k	Number of shards
D	Model dimensionality (number of trainable parameters)
ΔW_i	Local update of client i, $\Delta W_i \in \mathbb{R}^D$
π_i	Zero-knowledge proof for client i
C_i	Commitment for client i's update
$P_i(x)$	Polynomial proxy for client i's update
M_i	The wire message sent by client i to its shard chief
$\lvert M_i \rvert$	Size, in bytes, of message M_i
T	Number of training rounds

3.2 Message Composition in the Base System

In each round, the message a client transmits to its shard chief is

$$M_i = \big(\Delta W_i,\ \pi_i,\ C_i,\ P_i(x)\big). \tag{3.2}$$

Only ΔW_i scales with the model dimensionality D; the remaining components are of constant or near-constant size regardless of D, since the proof size, the commitment size, and the number of polynomial coefficients are all fixed by construction and do not grow with the number of model parameters. Consequently,

$$|M_i| \ \approx\ \underbrace{c_1 \cdot D}_{\Delta W_i} + \underbrace{c_2}_{\pi_i + C_i + P_i(x)}, \qquad c_1 \cdot D \gg c_2 \text{ for models of realistic size,} \tag{3.3}$$

where c_1 is the per-parameter encoding cost and c_2 is the small, constant cryptographic overhead. This asymptotic dominance of the ΔW_i term is the formal statement of the communication bottleneck identified in Section 2, and it holds for any N, k, or D.

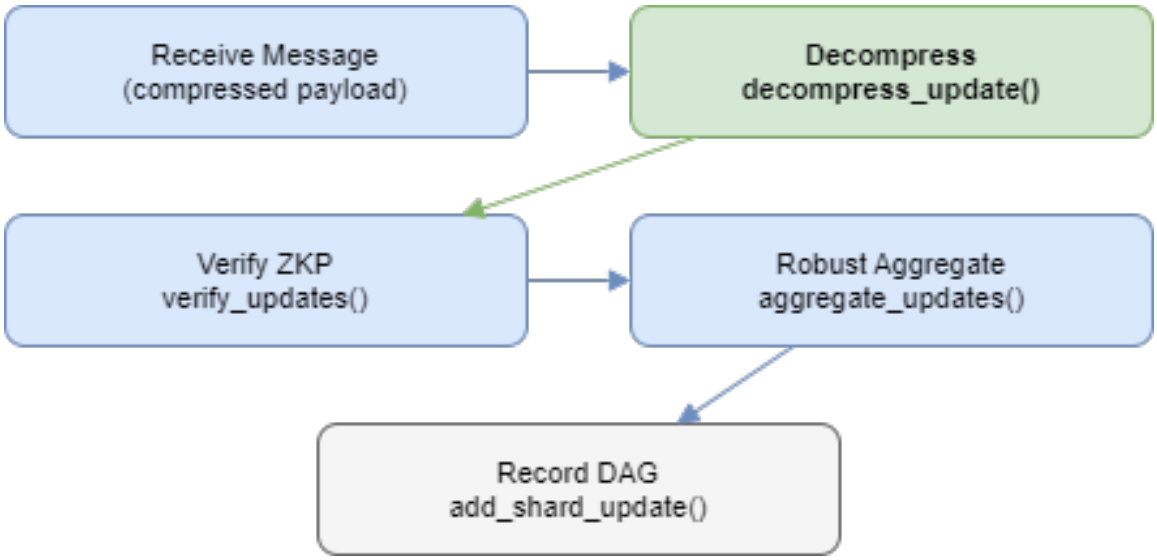


Figure 3.4: Shard-chief side processing pipeline, showing how the shard chief receives updates, decompresses them if compression was applied, verifies proofs, and feeds the reconstructed dense arrays into the robust aggregation subsystem. Decompression is the first step, ensuring that verification and aggregation remain unmodified and continue to operate on full-dimensional arrays as designed.

3.3 Total Communication Volume

Aggregating over all clients and the upward shard-to-global hop, the total communication volume of a single round is

$$V_{\text{round}} = \underbrace{\sum_{i=1}^{N} |M_i|}_{\text{client}\to\text{shard chief}} + \underbrace{k \cdot |W_s|}_{\text{shard chief}\to\text{global}} + \underbrace{N \cdot |W_t|}_{\text{global}\to\text{clients (broadcast)}}, \tag{3.4}$$

and over the full training run,

$$V_{\text{total}} = T \cdot V_{\text{round}}. \tag{3.5}$$

Since $|W_s|$ and $|W_t|$ are also full model vectors of dimensionality D, all three terms scale with D; however, the client-to-shard-chief term dominates in message *count*, since it involves N messages compared to only k messages on the upward hop, and $k \leq N$ by construction of the sharded topology. This makes the client-to-shard-chief hop the primary target for optimization, independently of the specific values of N and k.

3.4 Message Count

In addition to volume, the number of discrete messages exchanged per round is

$$\#\text{messages}_{\text{round}} = \underbrace{N}_{\text{client}\to\text{shard}} + \underbrace{k}_{\text{shard}\to\text{global}} + \underbrace{N}_{\text{broadcast}}. \tag{3.6}$$

This thesis's contribution targets *volume* per message, $|M_i|$, and leaves message *count* unchanged; this scope boundary is stated explicitly in Section 4.2.

3.5 Requirements for the Proposed Architecture

The communication optimization proposed in this thesis must satisfy *all* of the following requirements jointly, not as alternative design choices but as a single set of constraints derived directly from the guarantees already established by the base system. Each requirement addresses a distinct existing guarantee that the optimization must not break:

R1 — Compatibility with verification. The optimization must not invalidate the existing zero-knowledge verification pipeline. In particular, the norm-bound check and the polynomial-proxy consistency check performed during verification must remain meaningful.

R2 — Compatibility with integrity checking. The commitment scheme must continue to bind the shard chief to a verifiable representation of what was actually transmitted.

R3 — Compatibility with robust aggregation. The existing multi-stage robust aggregation procedure must continue to operate correctly; in particular, the update handed to the aggregation routine must be a full-dimension, dense numerical array, as required by the existing implementation.

R4 — Boundedness of accuracy loss. Any lossy transformation of the transmitted update must be evaluated against the base system's existing accuracy results, in order to bound the degradation introduced.

R5 — Measurability. The optimization must expose real, measurable communication statistics (bytes transmitted, compression ratio) rather than architectural estimates, addressing the absence of a real transmission boundary noted in Section 2.

R6 — Non-interference with the DAG layer. Since the DAG only stores a model hash and a commitment for each update (Section 2), the optimization must not require any change to the DAG module.

Section 4 shows, requirement by requirement, how the single proposed architecture satisfies each of R1–R6 simultaneously.

4 Proposed Communication-Enhanced Architecture

4.1 Overview and Design Rationale

Section 3 established that the transmitted update $\Delta W_i \in \mathbb{R}^D$ dominates communication cost (Eq. 3.3), while the cryptographic components (π_i, C_i, $P_i(x)$) are comparatively negligible, regardless of the specific scale of the deployment. Any communication optimization is therefore most effective when it targets ΔW_i directly, rather than the already-small cryptographic overhead surrounding it. We accordingly propose **gradient compression at the client-to-shard-chief boundary**: a new compression layer that reduces the size of the transmitted representation of ΔW_i while leaving the verification and aggregation pipelines — and the guarantees they provide — intact.

This is the central contribution of this thesis. Rather than modifying the verification, aggregation, or coordination logic of the base system, the proposed architecture is deliberately confined to the two points at which ΔW_i crosses a component boundary: its

encoding on the client before transmission, and its reconstruction on the shard chief before it re-enters the existing pipeline. This confinement is a design choice, not an incidental property: it is what allows the optimization to satisfy all six requirements of Section 3.5 simultaneously, since every existing verification and aggregation routine is left unmodified in its internal logic and continues to operate on exactly the kind of object it was designed for.

The design rationale can thus be summarized as a single principle: communication cost is reduced by changing *what is transmitted*, never by changing *what is verified, aggregated, or recorded*. The remainder of this section specifies this principle concretely, in terms of scope (Section 4.2), architectural components (Section 4.3), the compression procedure itself (Section 4.4), and the precise guarantees this construction preserves (Sections 4.5–4.6).

4.2 Scope

This thesis targets the **client-to-shard-chief** communication hop, which involves the largest number of concurrent messages of any hop in the system (one per client, per round) and constitutes the largest fan-in point in the architecture. The shard-chief-to-global hop involves a much smaller number of messages, already reduced from a number proportional to the total client count to a number proportional to the shard count by the sharded topology itself; it is therefore treated as an optional extension and left for future work, since it is architecturally analogous but of smaller marginal benefit. Optimizing message *count* (as opposed to per-message *volume*) is likewise out of scope and is discussed as future work at the end of this chapter.

4.3 Architectural Components

Table 3.2 summarizes the existing, modified, and newly introduced components of the proposed architecture.

4.4 Compression Procedure

We adopt top-magnitude sparsification with client-side *error feedback* (residual accumulation), optionally combined with fixed-point quantization. Error feedback is standard practice in the sparsified/quantized-SGD literature: rather than discarding non-transmitted coordinates permanently, it accumulates them into a residual buffer and re-injects them in later rounds, bounding the long-run information loss that plain top-k sparsification would otherwise introduce — a concern that is particularly relevant under non-independent-and-identically-distributed (non-IID) client data.

Figure 3.5 provides a detailed visual breakdown of the compression algorithm, illustrating each stage from gradient input through sparse encoding, quantization, and serialization metadata.

Procedure 1 — Compress Update.

1. Maintain a residual buffer $r_i \in \mathbb{R}^D$ per client, initialized to $\mathbf{0}$ and persisted across rounds.

Table 3.2: Components of the proposed architecture, existing versus new.

Component	**Status**	**Description**
Update Computation Module	Existing	Computes and clips the local update (unchanged)
Polynomial Proxy Generator	Existing	Computes the proxy from the *original* update (unchanged)
Zero-Knowledge Proof Generator	Existing	Generates the proof from the *original* update (unchanged)
Compression Module	**New**	Produces a compressed representation of the update, combining error feedback and top-magnitude sparsification (Section 4.4)
Residual/Error-Feedback Buffer	**New**	Client-side state accumulating non-transmitted error across rounds, to bound information loss (Procedure 1)
Commitment Generator	Modified	Now computes the commitment over the *compressed* representation, after proof generation (Section 4.5)
Peer-to-Peer Communication Module	Existing	Unaffected; carries whatever payload it is handed
Decompression Module	**New**	Reconstructs a full-size dense array before verification and aggregation
Proof Verifier	Existing	Verifies the proof against the original-update-derived proxy (unchanged)
Robust Aggregation Module	Existing	Operates on the reconstructed dense array (unchanged)
Communication Metrics Logger	**New**	Records real, serialized byte counts per round

2. Form the error-compensated update $\Delta W_i' = \Delta W_i + r_i$ from the current round's clipped update ΔW_i.

3. Retain the $k_{\text{ret}} = \max(1, \lfloor \rho \cdot D \rfloor)$ largest-magnitude coordinates of $\Delta W_i'$, giving index set $\mathcal{I}$ and values $v = \Delta W_i'[\mathcal{I}]$.

4. Update the residual: $r_i \leftarrow \Delta W_i'$, then $r_i[\mathcal{I}] \leftarrow 0$.

5. If quantization is enabled with bit-width b, map each retained value to an integer level via Eq. (3.7).

6. Output $(\mathcal{I}, v \text{ or } q, D)$, together with $(v_{\min}, v_{\max}, b)$ if quantization was applied.

$$q = \text{round}\left(\frac{v - v_{\min}}{v_{\max} - v_{\min}} \cdot (2^b - 1) \right), \qquad v_{\min} = \min(v),\ v_{\max} = \max(v). \tag{3.7}$$

Procedure 2 — Decompress Update.

1. Initialize $\widehat{\Delta W} = \mathbf{0} \in \mathbb{R}^D$.

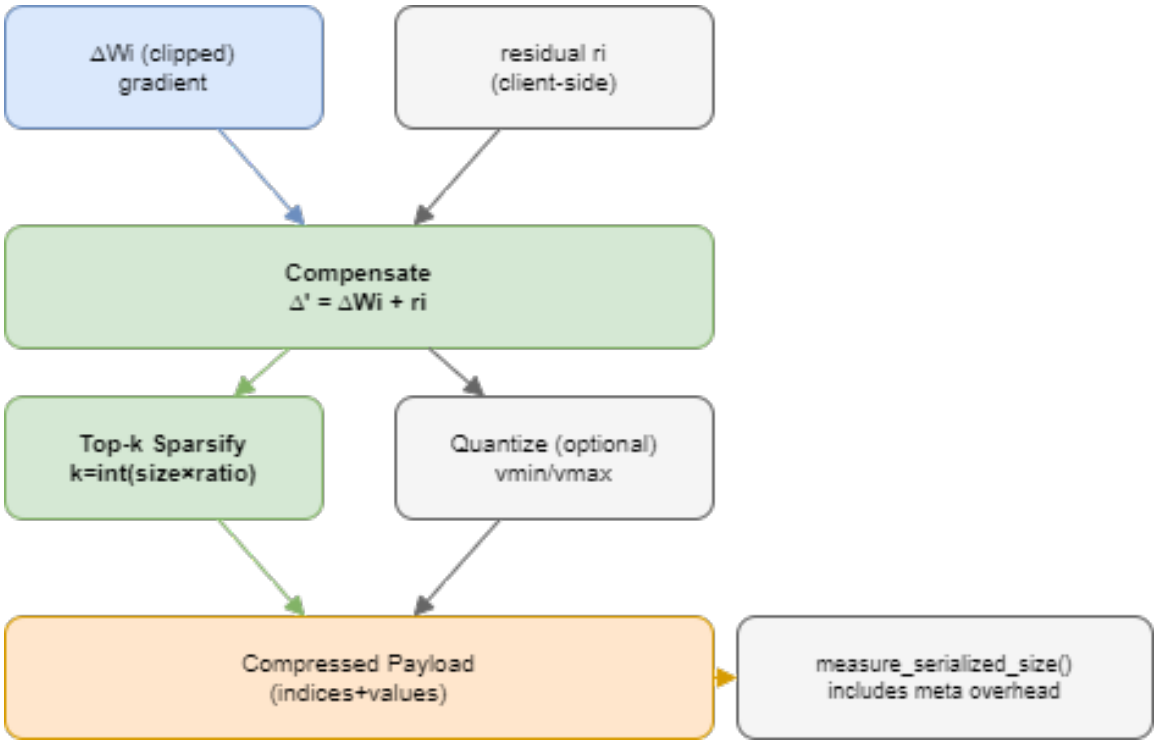


Figure 3.5: Compression procedure showing the step-by-step transformation of a full-dimensional gradient into a compressed sparse representation. The procedure includes residual compensation (blending the current gradient with accumulated error from prior rounds), top-k magnitude retention using the formula $k = \lfloor \rho \cdot D \rfloor$, optional quantization to reduce per-value bit-width, and packing of indices, values, and metadata for transmission. The serialized size includes all components: retained indices, quantized values, and parameter-size information.

2. If quantization was applied, invert Eq. (3.7) to recover v.

3. Set $\widehat{\Delta W}[\mathcal{I}] \leftarrow v$ and return $\widehat{\Delta W}$.

The residual buffer r_i is client-side-only state, never transmitted; Procedure 2 is therefore identical whether or not error feedback is enabled. The reconstruction $\widehat{\Delta W}_i$ is exact on $\mathcal{I}$ (up to quantization error) and zero elsewhere, and is what the unmodified robust aggregation pipeline receives, satisfying requirement R3. The quantitative effect of error feedback on convergence is evaluated empirically in Chapter 4.

4.5 Placement in the Client and Shard-Chief Pipelines

Client side. Processing follows a fixed order: (1) the update is computed and clipped to the norm bound; (2) the polynomial proxy and the zero-knowledge proof are computed from this clipped, uncompressed update, since both certify a claim about the true computed update that must hold independently of its later encoding; (3) only afterwards is the update passed through Procedure 1; (4) the commitment is computed over the resulting compressed payload, so that it binds the receiver to exactly what is transmitted (requirement R2). Sparsification and quantization are therefore strictly post-proof encoding steps.

Shard-chief side. Decompression (Procedure 2) is the first step of the shard chief's round-processing routine, before verification and aggregation, both of which continue to receive full-dimension dense arrays unchanged (requirement R3). The residual buffer is client-side-only and plays no role at this stage.

4.6 ZKP–Commitment Soundness Argument

Since π_i is generated over the original update while C_i is computed over the compressed payload, nothing in the base pipeline inherently links the two: a client could, in principle, submit a compressed payload derived from an unproven vector. We close this gap by extending the public statement verified alongside π_i to additionally assert that the transmitted compressed payload equals the output of Procedure 1 applied to the same ΔW_i used for the norm-bound and proxy checks.

> **Invariant I.** Any compressed payload accepted by the shard chief is a deterministic compression of an update that itself satisfies the base system's norm-bound and proxy-consistency guarantees.

This preserves requirement R1 and requires no change to the DAG module (requirement R6). The fully rigorous realization would embed this equality directly inside the zero-knowledge circuit; as a practical alternative, consistent with the trust model already used elsewhere in the base system, we instead have the client authenticate the tuple (compressed payload, ρ, b) and rely on the existing reputation mechanism to spot-check compression correctness on flagged clients. In-circuit enforcement is left as future work.

4.7 Communication Complexity Analysis

Under top-magnitude sparsification with retention ratio ρ (ignoring quantization for a first-order estimate), each retained coordinate requires an index and a value, so

$$|M_i^{\text{new}}| \;\approx\; \underbrace{\rho D \cdot (c_{\text{idx}} + c_{\text{val}})}_{\Delta W_i \text{ compressed}} + c_2, \tag{3.8}$$

compared to the base message size of Eq. (3.3). For $\rho \ll 1$, with $(c_{\text{idx}} + c_{\text{val}})$ on the order of c_1, this gives an asymptotic reduction on the order of $1/\rho$ in the dominant term, independently of N, k, or D; quantization further reduces c_{val}, compounding the reduction.

Illustrative example. For a hypothetical $D = 10^5$ (chosen purely for illustration) with $c_1 = 4$ bytes, the base system transmits $\approx 4 \times 10^5$ bytes per client per round. At $\rho = 0.1$ with 4-byte indices and values, Eq. (3.8) gives $\approx 8 \times 10^4$ bytes (a fivefold reduction); adding 8-bit quantization reduces this further to $\approx 5 \times 10^4$ bytes (an eightfold reduction). These figures are illustrative only; actual values for this system are reported in Chapter 4.

Total round volume becomes

$$V_{\text{round}}^{\text{new}} \;=\; \sum_{i=1}^{N} |M_i^{\text{new}}| + k|W_s| + N|W_t|, \tag{3.9}$$

with the shard-to-global and broadcast terms unchanged under the scope of Section 4.2. Message *count* is unaffected: this architecture reduces $|M_i|$, not the number of messages exchanged per round; reducing count would require a different class of optimization (e.g., a topology change), left as future work.

4.8 Communication Metrics Logger

To satisfy requirement R5 despite the absence of a real transmission boundary in the base system (Section 2), a logging component *actually serializes* both the original and compressed representations at the client-to-shard-chief boundary and records the resulting byte counts, compression ratio, and bytes saved per round, integrating with the base system's existing per-round metrics structure.

4.9 Compatibility with Robust Aggregation under Sparse Reconstructions

The existing robust aggregation stages were designed and evaluated in Chapter 2 against dense updates. Under the proposed architecture, the shard chief instead receives reconstructions $\widehat{\Delta W}_i$ that are exact on a retained subset and zero elsewhere (Procedure 2). We argue this remains compatible with the existing defenses for two reasons: (i) magnitude-based retention preferentially preserves the coordinates carrying the most discriminative signal, on which both cluster consensus and reputation weighting rely; (ii) since all clients in a shard share the same ρ per round, the zero-filled pattern is structurally common across reconstructions and does not bias any single client. This is an analytical argument, not a proof — an adversary could in principle target coordinates likely to be retained — and its empirical confirmation is left to Chapter 4.

4.10 Trade-offs

Table 3.3 summarizes the main advantages and disadvantages of the proposed compression layer.

Table 3.3: Trade-offs of the proposed compression layer.

Advantages	**Disadvantages**
Directly targets the dominant term in $\|M_i\|$ (the update itself), rather than the already-small cryptographic overhead	Lossy: introduces an accuracy-versus-communication trade-off that must be empirically bounded
Leaves proof generation and verification, the polynomial proxy, robust aggregation, and the DAG entirely untouched at the logic level	Requires an explicit design decision on what the commitment binds to (Section 3.5, R2)
Localized changes, confined to the client and shard-chief modules and one new compression module	Does not reduce message count, nor address the shard-to-global or broadcast hops (explicit scope boundary)
Compatible with the existing norm-bound proof, since the proof is computed on the original update, strictly before compression (Section 4.5)	The practical realization of the ZKP–commitment soundness invariant (Section 4.6) relies on authentication and spot-checking rather than full in-circuit enforcement, which is left as future work
Error feedback (Section 4.4) bounds long-run information loss from non-retained coordinates, addressing a known limitation of plain top-k sparsification	Robustness of the reputation and clustering defenses under sparse reconstructions is argued analytically (Section 4.9) but still requires empirical confirmation in Chapter 4

Conclusion

This chapter formalised the communication behaviour of the existing system, showed that the full-dimension transmission of the local update dominates communication cost (Eq. (3.3)), and proposed a gradient compression layer at the client-to-shard-chief boundary as this thesis's Communication-Enhanced Architecture. The model and the proposed design were expressed in general terms — client count, shard count, model dimensionality — independently of any specific deployment scale.

Beyond the core compression mechanism, the chapter equipped it with client-side error feedback to bound long-run information loss; fixed the processing order between proof generation and compression; closed a soundness gap between the zero-knowledge proof and the commitment through an extended public statement (Invariant I); and argued, while flagging the need for empirical confirmation, that the existing robust aggregation defenses should remain effective under sparse reconstructions. All six requirements of Section 3.5 are satisfied by construction: verification and aggregation logic remain untouched, the commitment scheme is redefined consistently with what is transmitted, and a new metrics logger exposes real, measurable communication statistics.

Chapter 4 details the implementation of this design and reports its experimental evaluation: the compression–accuracy trade-off across retention ratios, the contribution of error feedback, behavior under non-IID and asynchronous conditions, and the robustness validation identified in Section 4.9.

Chapter 4

Implementation — Communication Optimization

1 Overview and positioning

This chapter documents the concrete implementation work produced as the master's contribution to the baseline PFE project. It presents the realization of the architecture proposed in Chapter 3, specifically the compression layer with integrated residual/error-feedback mechanism, the explicit requirements mapping (R1–R6), and the soundness argument underlying ZKP and commitment binding.

This chapter purposefully avoids re-stating the baseline system architecture described in Chapter 3. Rather, it focuses exclusively on the incremental artifacts introduced by this contribution. The reader should consult Chapter 3 for background on the federated learning protocol, proof semantics, and aggregation mechanisms.

2 Design goals and implementation constraints

The implementation was driven by three pragmatic goals:

1. Measurability: introduce an explicit serialization boundary so that transmitted bytes can be defensibly counted in the local simulation.

2. Non-intrusiveness: preserve the existing ZKP and aggregation semantics so the new layer does not require re-designing the verification pipeline.

3. Error-feedback support: implement residual/error-feedback accumulation as defined in Chapter 3 Procedure 1 to recover accuracy under aggressive compression.

Constraints that shaped implementation choices:

- Time: changes must be achievable within a single-master timeline and must not require redesign of the ZKP machinery.

- Backwards compatibility: the system must run unchanged when compression is disabled (ratio=1.0).

- Minimal external dependencies: prefer small, auditable code rather than introducing heavy compression libraries.

3 High-level implementation summary

The implementation adds a single new module (`gradient_compression.py`), augments client state with a residual buffer, and wires measured payloads into the shard processing path and global instrumentation. The functional decomposition is intentionally narrow and orthogonal to the baseline: compression is a transport concern that sits between client-local computation and shard-level verification.

Key engineering artifacts produced:

- `gradient_compression.py`: compression module with integrated residual compensation, supporting top-k sparsification and quantization.
- Client residual buffer (`client.py`): per-client state for error-feedback accumulation across rounds.
- Client changes (`client.py`): apply residual compensation, compress the compensated update, update residual, and record metadata.
- Shard chief changes (`shard_chief.py`): accept optional compression metadata and decompress payloads before verification and aggregation.
- System instrumentation (`federated_learning_system.py`): collect per-client and per-round communication metrics.
- Configuration (`federated_learning_config.py`): new constants for method, ratio and quantize bits.

4 Detailed implementation

4.1 gradient_compression.py — Module interface with residual support

The new module exposes two primary functions:

compress_update_with_residual(gradient, residual, method, ratio, quantize_bits)
Implements Procedure 1 from Chapter 3. Returns a triple `(payload, meta, new_residual)`.

1. Compensates the gradient: $\Delta W_i' := \Delta W_i + r_i$
2. Compresses $\Delta W_i'$ using the kernel function (see below)
3. Updates the residual: $r_i := \Delta W_i'$ with retained coordinates zeroed
4. Returns the compressed payload and updated residual for the next round

The residual is client-side only and never transmitted; only the compressed payload is sent to the shard chief.

compress_update_kernel(gradient, method, ratio, quantize_bits) Core compression engine supporting three methods: `topk`, `quantize`, and `topk_quantize`. For `topk`, selects the $k = \lceil rd \rceil$ largest-magnitude entries and returns their indices and values. For `quantize`, applies linear scaling to an integer bit-width range and stores min/max for dequantization. The returned metadata contains original shape, dtype, indices (if sparsified), and quantization parameters (if applied).

measure_serialized_size(obj) Uses Python's `pickle.dumps` to measure real serialized bytes. This approach avoids analytical estimates and captures the overhead of metadata encoding, ensuring defensible numbers for the evaluation chapter.

Implementation notes:

- Indices are stored as `np.int32` to minimize serialized size and ensure cross-platform consistency.
- Quantized arrays use `np.uint8` (8-bit) or `np.uint16` (16-bit) depending on `quantize_bits`.
- The residual buffer is always dense and tracks the full parameter dimension; this simplifies accounting and allows the verifier to skip decompression steps in principle (though currently decompression always occurs for uniformity).

4.2 Client-side changes (client.py) — residual state and compressed workflow

Where: `FederatedLearningClient` class.
New instance variable:

`self.residual` A dense `np.ndarray` of shape matching the flattened parameter vector, initialized to zeros at client startup. It persists across training rounds and stores the accumulated compression error for error-feedback.

Modified sequence in `create_update(...)`:

1. Compute the clipped gradient ΔW_i as before (unchanged).
2. Call `compress_update_with_residual(gradient, self.residual, ...)` to apply Procedure 1:
   - Compensate: $\Delta W_i' := \Delta W_i + r_i$
   - Compress $\Delta W_i'$
   - Obtain r_i^{new} (residual for next round)
3. Update client state: `self.residual = new_residual`
4. Measure real serialized sizes for the original gradient and the tuple `(payload, meta)`.
5. Compute SHA-256 commitment over the transmitted payload (the compressed $\Delta W_i'$ or its sparsified representation).
6. Generate polynomial proxy and ZKP from the original (uncompressed) ΔW_i for soundness preservation.
7. Populate the `ClientUpdate` object with gradient, compressed payload, compression metadata, and ZKP.

Rationale:

- Error-feedback recovers accuracy by accumulating information that was dropped during sparsification. This is a well-established technique in distributed SGD with compression.
- The residual is never transmitted, only stored client-side, so it does not increase communication. It is conceptually internal to the client's compression algorithm.
- Proofs remain tied to the original gradient, preserving the cryptographic model from the baseline system.

4.3 Shard-side changes (shard_chief.py)

Where: `ShardChief.process_round(...)` signature extended to accept an optional `compression_metas` list.

Operational changes:

- If `compression_metas` is provided, decompress each incoming payload via `decompress_update` before calling existing `verify_updates` and aggregation routines.
- The rest of the verification and aggregation code is unchanged.

This minimal change preserves experimental control: shard-level verification semantics do not change.

4.4 System instrumentation (federated_learning_system.py)

Changes to collect and record communication metrics:

- During client update collection, extract and record `compression_meta['original_size_bytes']` and `compression_meta['compressed_size_bytes']`.
- After all clients have submitted, compute and insert a `communication_stats` block into `round_meta`:
  - Total bytes original, total bytes compressed
  - Average compression ratio
  - Absolute bytes saved
- Persist into system metrics so the information is available in the JSON summary for post-processing.

5 Mapping to Requirements R1–R6

Chapter 3 defines six formal requirements for contibution. The implementation addresses each as follows:

Requirement	**How implemented**
R1: Verification	Proofs continue to be generated over the original dense gradient. Shard chief decompresses before verification, so the verifier receives arrays identical to the baseline.
R2: Integrity	SHA-256 commitment computed over the transmitted payload (compressed indices and values). DAG and audit logs refer to the actual transmitted object.
R3: Aggregation	Compressed payloads are decompressed to full-size arrays before the robust aggregation pipeline; aggregation logic is unchanged.
R4: Accuracy	Experimental sweeps (Chapter 5) report model accuracy and F1 with and without compression. Error-feedback helps recover accuracy under sparsification.
R5: Measurability	Real serialized byte counts via `pickle`-based measurement. Includes payload and metadata overhead.
R6: DAG recording	No change to DAG logic. Commitment now binds to compressed payload; this aligns audit logs with actual transmission.

Table 4.1: Mapping of implementation choices to formal requirements R1–R6.

6 ZKP-Commitment soundness and Invariant I

Chapter 3 Section 4.6 introduces Invariant I: *any compressed payload accepted by the shard chief is a deterministic compression of an update that already satisfied the norm-bound and proxy-consistency checks.*

Practical realisation chosen in this implementation:

1. The client computes the proof π_i over the original (uncompressed) gradient, which by construction is clipped to the norm bound and used to build the polynomial proxy.

2. The commitment is computed over the transmitted payload (compressed) to bind the audit trail to what was actually sent.

3. The shard chief, upon decompression, relies on the existing reputation-based spot-check mechanism (from the robust aggregation layer) to validate that the decompressed update is a plausible gradient. If an update is maliciously corrupted during compression, it will be flagged as an outlier during clustering and downweighted accordingly.

4. Full in-circuit enforcement (verifying that the compressed payload is a valid compression of the committed original) is left for future work, as it would require an

additional ZKP or a secondary cryptographic commitment.

This design preserves the soundness of the baseline system while minimizing implementation complexity within the master's timeline.

7 Testing and correctness checks

Correctness validation includes:

1. Unit test: `test_compression_kernel` verifies that applying `decompress_update` after `compress_update_kernel(x)` reconstructs the original vector at retained positions, and that quantized values are dequantized within a specified numeric tolerance.

2. Residual test: `test_residual_accumulation` verifies that the residual buffer is correctly updated and that repeated compression with residual converges to zero error for noiseless updates.

3. Sanity run: Execute a single training round with `COMPRESSION_RATIO=1.0` and compare shard aggregate norms against the unmodified baseline to verify identical numeric outputs.

4. Integration run: Run the full 5-round experiment with and without compression; verify that `communication_stats` appears in JSON output and that global aggregation completes successfully.

All tests are lightweight and executable on a commodity laptop.

8 Experimental driver and reproducible workflow

A simple driver script (`run_compression_sweep.py`) orchestrates parameter sweeps:

1. Iterates over specified compression ratios and methods.

2. Sets configuration constants programmatically.

3. Runs `execute_full_training()` for each configuration and writes JSON artifacts to `experiments/`.

4. Aggregates CSV summaries for plotting: total bytes per run, final F1, verification success rate, and per-round elapsed times.

The driver includes a flag `--enable-residual` (default: true) to allow ablation studies comparing error-feedback enabled vs. disabled.

9 Security, auditability and ethical considerations

Binding commitments to compressed payloads introduces subtle security considerations:

- Audit interpretation: DAG entries refer to the compressed object. An auditor must know the compression method and metadata to reconstruct the transmitted object.

- Integrity vs. semantic binding: committing to the compressed payload ensures integrity of transmitted bits but not of the original gradient. The practical realisation relies on reputation-based spot-checking for validation.
- Privacy: top-k sparsification leaks which parameters changed most (via index patterns). The thesis (Chapter 6) discusses this trade-off; practitioners should evaluate privacy impact before deployment.
- Error-feedback implications: the residual buffer accumulates dropped information on the client. In scenarios requiring parameter audits, this introduces a non-trivial gap between the transmitted and true underlying gradient.

10 Limitations and planned extensions

This implementation is an engineering iteration. Key planned extensions:

- Entropy coding and delta-encoding to further reduce payload size while maintaining determinism.
- Networked deployment: port the serialization layer to actual P2P transport (TCP/UDP, protobuf framing) to measure wall-clock latency effects.
- Formal in-circuit contracts: add a secondary ZKP proving that the compressed payload is a valid lossy encoding of the original gradient. This strengthens the soundness argument beyond the practical reputation-based check.
- Adaptive compression ratios: tighter early in training (when updates are large), looser later (when they converge), potentially coordinated with learning rate schedules.

11 Practical instructions for reproducing results

To reproduce experiments from Chapter 5:

1. Install dependencies. Python 3.8+ and standard packages (numpy, scikit-learn) are sufficient.
2. Run baseline (no compression): `python run_compression_sweep.py --ratios 1.0 --rounds 5`
3. Run compression sweeps: `python run_compression_sweep.py --methods topk --ratios 0.5 0.2 0.1 0.05 --rounds 5`
4. Ablation study (no error-feedback): `python run_compression_sweep.py --methods topk --ratios 0.1 --enable-residual false`
5. Aggregated artifacts will be written to `experiments/`. The plotting notebook is located at `notebooks/plot_comm_vs_accuracy.ipynb`.

12 Conclusion

The contribution implements a compact, auditable compression layer faithful to the architecture proposed in Chapter 3. It integrates residual/error-feedback accumulation at the client, preserves verifier semantics via proof generation over the original gradient, and provides defensible byte accounting via real serialization. The implementation prioritises reproducibility and auditability. It is intentionally conservative in scope, focusing on core feasibility within a master's timeline while documenting clear roadmaps for future extensions such as formal in-circuit contracts and networked deployment.

Bibliography

[1] J. C. Duchi, A. Agarwal, and M. J. Wainwright, "Dual averaging for distributed optimization," in *Proceedings of the 50th Annual Allerton Conference on Communication, Control, and Computing*, 2012, pp. 1564–1565. DOI: 10.1109/ALLERTON.2012.6483406

[2] A. Nedić and A. Ozdaglar, "Distributed subgradient methods for multi-agent optimization," *IEEE Transactions on Automatic Control*, vol. 54, no. 1, pp. 48–61, 2009. DOI: 10.1109/TAC.2008.2009515

[3] X. Lian, C. Zhang, H. Zhang, C.-J. Hsieh, W. Zhang, and J. Liu, "Can decentralized algorithms outperform centralized algorithms? A case study for decentralized parallel stochastic gradient descent," in *Advances in Neural Information Processing Systems*, vol. 30, 2017. [Online]. Available: https://proceedings.neurips.cc/paper/2017/hash/6c340f25839e6acdc73414517203f5f0-Abstract.html

[4] H. B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. A. y Arcas, "Communication-efficient learning of deep networks from decentralized data," in *Proceedings of the 20th International Conference on Artificial Intelligence and Statistics*, ser. Proceedings of Machine Learning Research, vol. 54, PMLR, 2017, pp. 1273–1282. [Online]. Available: https://proceedings.mlr.press/v54/mcmahan17a.html

[5] J. Konečný, H. B. McMahan, F. X. Yu, P. Richtárik, A. T. Suresh, and D. Bacon, "Federated learning: Strategies for improving communication efficiency," *arXiv preprint arXiv:1610.05492*, 2016. [Online]. Available: https://arxiv.org/abs/1610.05492

[6] S. U. Stich, "Local SGD converges fast and communicates little," in *International Conference on Learning Representations*, 2019. [Online]. Available: https://openreview.net/forum?id=S1g2JnRcFX

[7] E. T. M. Beltrán et al., "Decentralized federated learning: Fundamentals, state-of-the-art, frameworks, trends, and challenges," *arXiv preprint arXiv:2211.08413*, 2022. [Online]. Available: https://arxiv.org/abs/2211.08413

[8] H. Ye, L. Liang, and G. Y. Li, "Decentralized federated learning with unreliable communications," *IEEE Journal of Selected Topics in Signal Processing*, vol. 16, no. 3, pp. 487–500, 2022. DOI: 10.1109/JSTSP.2022.3152445

[9] S. Boyd, A. Ghosh, B. Prabhakar, and D. Shah, "Randomized gossip algorithms," *IEEE Transactions on Information Theory*, vol. 52, no. 6, pp. 2508–2530, 2006. DOI: 10.1109/TIT.2006.874516

[10] Q. Chen, Z. Wang, H. Wang, and X. Lin, "FedDual: Pair-wise gossip helps federated learning in large decentralized networks," *IEEE Transactions on Information Forensics and Security*, vol. 18, pp. 335–350, 2023. DOI: 10.1109/TIFS.2022.3222935

[11] X. Lian, W. Zhang, C. Zhang, and J. Liu, "Asynchronous decentralized parallel stochastic gradient descent," in *Proceedings of the 35th International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, vol. 80, PMLR, 2018, pp. 3043–3052. [Online]. Available: https://proceedings.mlr.press/v80/lian18a.html

[12] S. Lachapelle, P. Brouillard, T. Deleu, and S. Lacoste-Julien, "Gradient-based neural DAG learning," in *International Conference on Learning Representations*, 2020. [Online]. Available: https://openreview.net/forum?id=rklbKA4YDS

[13] D. Alistarh, D. Grubic, J. Li, R. Tomioka, and M. Vojnovic, "QSGD: Communication-efficient SGD via gradient quantization and encoding," in *Advances in Neural Information Processing Systems*, vol. 30, 2017. [Online]. Available: https://arxiv.org/abs/1610.02132

[14] L. Chen, W. Liu, Y. Chen, and W. Wang, "Communication-efficient design for quantized decentralized federated learning," *arXiv preprint arXiv:2303.08423*, 2023. [Online]. Available: https://arxiv.org/abs/2303.08423

[15] S. U. Stich, J.-B. Cordonnier, and M. Jaggi, "Sparsified SGD with memory," in *Advances in Neural Information Processing Systems*, vol. 31, 2018. [Online]. Available: https://proceedings.neurips.cc/paper/2018/hash/bc6dc48b743dc5d013b1abaebd2faed2-Abstract.html

[16] A. Koloskova, T. Lin, S. U. Stich, and M. Jaggi, "Decentralized deep learning with arbitrary communication compression," in *International Conference on Learning Representations*, 2020. [Online]. Available: https://openreview.net/forum?id=SkgGCkrKvH

[17] D. Weissteiner, "Decentralized federated learning: Framework design, communication efficiency, and dynamic synchronization," 2025. [Online]. Available: https://ywcb00.ywcb.org/assets/pdf/Weissteiner2025_DecentralizedFederatedLearning_MasterThesis.pdf

[18] Y. Kai, "Strategies for improving communication efficiency in distributed and federated learning: Compression, local training, and personalization," 2025. DOI: 10.25781/kaust-80584

[19] W. Li, T. Lv, W. Ni, J. Zhao, E. Hossain, and H. V. Poor, "Decentralized federated learning over imperfect communication channels," 2024. [Online]. Available: https://arxiv.org/abs/2405.12894

[20] E. Jeong and M. Kountouris, "Draco: Decentralized asynchronous federated learning over row-stochastic wireless networks," *IEEE Open Journal of the Communications Society*, vol. 6, pp. 4818–4839, 2024. DOI: 10.1109/OJCOMS.2025.3574098

[21] H. Xie, M. Xia, P. Wu, S. Wang, and K. Huang, "Decentralized federated learning with asynchronous parameter sharing," 2023. DOI: 10.1109/ICCCWorkshops57813.2023.10233712

[22] C. A. S. H. Kaluarachchi and R. Buyya, "Asynchronous decentralized federated learning over lossy wireless links via reception- and age-aware aggregation," 2026. [Online]. Available: https://arxiv.org/abs/2606.10774v2

[23] H. Seferoglu, "Digest: Fast and communication efficient decentralized learning with local updates," 2023.

[24] Z. Tang, S. Shi, B. Li, and X. Chu, "Gossipfl: A decentralized federated learning framework with sparsified and adaptive communication," *IEEE Transactions on Parallel and Distributed Systems*, vol. 34, pp. 909–922, 2023. DOI: 10.1109/TPDS.2022.3230938

[25] F. Mulitze, H. Woisetschläger, and H.-A. Jacobsen, "Mar-fl: A communication efficient peer-to-peer federated learning system," 2025. DOI: 10.48550/arXiv.2512.05234

[26] "When decentralized optimization meets federated learning," 2023. DOI: 10.48550/arXiv.2306.02570

[27] "Private node selection in personalized decentralized learning," 2023. DOI: 10.48550/arXiv.2301.12755

[28] S. Kharrat, M. Canini, and S. Horvath, "Decentralized personalized federated learning," 2024. [Online]. Available: https://arxiv.org/abs/2406.06520

[29] "Communication-efficient federated learning via quantized compressed sensing," *IEEE Transactions on Wireless Communications*, vol. 22, no. 2, pp. 1087–1100, 2023. DOI: 10.1109/TWC.2022.3201207

[30] X. Fang, L. Chen, and H. Yin, "Qat-dfl: A communication efficient decentralized federated learning with quantization-aware training design," 2025. DOI: 10.1109/WCSP68525.2025.1010313

[31] C. Medjadji, S. Alawadi, F. M. Awaysheh, G. Leduc, S. Kubler, and Y. T. le Traon, "Fedsparq: Adaptive sparse quantization with error feedback for robust and efficient federated learning," 2025. DOI: 10.48550/arXiv.2511.05591

[32] Y. Xu et al., "Enhancing decentralized federated learning with model pruning and adaptive communication," *IEEE Transactions on Industrial Informatics*, 2024. DOI: 10.1109/TII.2024.3424497

[33] D. Kritsiolis and C. Kotropoulos, "Quantized rank reduction: A communications-efficient federated learning scheme for network-critical applications," 2025. DOI: 10.48550/arXiv.2507.11183

[34] P. M. S. Sánchez, E. T. M. Beltrán, M. L. Nistal, G. Bovet, G. M. Pérez, and A. H. Celdrán, "Profe: Communication-efficient decentralized federated learning via distillation and prototypes," 2024. DOI: 10.48550/arXiv.2412.11207

[35] "Impact analysis of delayed updates for decentralized federated learning: A delay-tolerant training strategy perspective for real-world edge intelligence," 2023. DOI: 10.21203/rs.3.rs-2595492/v1

[36] Y. Liao, Y. Xu, H. Xu, M. Chen, L. Wang, and C. Qiao, "Asynchronous decentralized federated learning for heterogeneous devices," *IEEE/ACM Transactions on Networking*, 2024. DOI: 10.1109/TNET.2024.3424444

[37] N. Masmoudi and W. Jaafar, "Ocd-fl: A novel communication-efficient peer selection-based decentralized federated learning," 2024. [Online]. Available: https://arxiv.org/abs/2403.04037

[38] M. R. Behera and S. Chakraborty, "Pfedgame – decentralized federated learning using game theory in dynamic topology," 2024. DOI: 10.1109/COMSNETS59351.2024.10427470

[39] R. Diestel, *Graph Theory*, 5th ed. Berlin, Germany: Springer, 2017. DOI: 10.1007/978-3-662-53622-3

[40] H. Nguyen, H.-T. Nguyen, P. K. Donta, S. Pirttikangas, and L. Lovén, "Graph-based gossiping for communication efficiency in decentralized federated learning," 2025. DOI: 10.48550/arXiv.2506.10607

[41] R. Zong and K. Li, "Fedcs: Efficient communication scheduling in decentralized federated learning," *Information Fusion*, 2023. DOI: 10.1016/j.inffus.2023.102028

[42] K. Attaluri, R. P. Diaz-Redondo, and M. F. Veiga, "Air-plan: Query-optimized topology selection for over-the-air decentralized federated learning," 2026. [Online]. Available: https://arxiv.org/abs/2607.04254v1

[43] "On the benefits of multiple gossip steps in communication-constrained decentralized federated learning," *IEEE Transactions on Parallel and Distributed Systems*, 2022. DOI: 10.1109/TPDS.2021.3138977

[44] W. Liu, L. Chen, and W. Zhang, "Decentralized federated learning: Balancing communication and computing costs," 2021.

[45] "Sparq-sgd: Event-triggered and compressed communication in decentralized optimization," *IEEE Transactions on Automatic Control*, vol. 68, no. 2, pp. 721–736, 2023. DOI: 10.1109/TAC.2022.3145576

[46] X.-F. Xue, H. Mao, Q. Li, F. Huang, and A. A. A. El-Latif, "An energy efficient specializing dag federated learning based on event-triggered communication," *Mathematics*, vol. 10, no. 22, p. 4388, 2022. DOI: 10.3390/math10224388

[47] M. Cao, B. Cao, W. Hong, Z. Zhao, X. Bai, and L. Zhang, "Dag-fl: Direct acyclic graph-based blockchain empowers on-device federated learning," 2021. DOI: 10.1109/ICC42927.2021.9500737

[48] Z. Shuaipeng et al., "Dag-afl: Directed acyclic graph-based asynchronous federated learning," 2025. DOI: 10.48550/arXiv.2507.20571

[49] J. Bin, Z. Bo, L. Fei, and S. Houbing, "Decentralized federated learning in metacomputing based on directed acyclic graph with optimized tip selector," 2025. DOI: 10.13016/m2vcth-sye7

[50] B. Jossekin, P. Bjarne, S. Robert, A. Bert, and P. Andreas, "Implicit model specialization through dag-based decentralized federated learning," 2022. DOI: 10.48550/arXiv.2111.01257

[51] Z. Zhai, X. Yuan, and X. Wang, "Decentralized federated learning via mimo over-the-air computation: Consensus analysis and performance optimization," *IEEE Transactions on Wireless Communications*, 2023. DOI: 10.1109/TWC.2024.3385443

[52] "A fast blockchain-based federated learning framework with compressed communications," *IEEE Journal on Selected Areas in Communications*, vol. 40, no. 12, pp. 3358–3372, 2022. DOI: 10.1109/JSAC.2022.3213345

[53] H. Wang and Y. Chi, "Communication-efficient federated optimization over semi-decentralized networks," 2023. DOI: 10.48550/arXiv.2311.18787

[54] R. Xiao, Y. Cao, and B. Xia, "Adaptive tip selection for dag-shard-based federated learning with high concurrency and fairness," 2024. DOI: 10.3390/s25010019

[55] Y. Liu, T. Zhang, Z. Shen, L. Wu, S. Chen, and J. Jin, "Towards heterogeneity-aware and energy-efficient topology optimization for decentralized federated learning in edge environment," 2025. [Online]. Available: https://arxiv.org/abs/2508.08278v1

[56] X. Huang and K. Yuan, "Optimal complexity in non-convex decentralized learning over time-varying networks," 2022. [Online]. Available: https://arxiv.org/abs/2211.00533v1

[57] P. Zhao and G. Zhang, "Decentralized federated learning towards communication efficiency, robustness, and personalization," *ACM Transactions on Sensor Networks*, 2025. DOI: 10.1145/3730587

[58] J. Chen, Y. Li, S. Sun, and X. Chen, "Dflchain: A dag-based dynamic federated learning framework with hybrid tip prioritization for trustworthy its data sharing," 2024.

[59] S. Yuan, B. Cao, M. Peng, and Y. Sun, "Chainsfl: Blockchain-driven federated learning from design to realization," 2021. DOI: 10.1109/WCNC49053.2021.9417299